CLICKUPFY

Guia completo do usuário

Planeje. Execute. Comprove.

Do ClickUp ao terminal e ao agente, com escopo, contexto e evidência.

Edição v2.0.0
Gerado em 13 de novembro de 2026
Fonte editorial: docs/user/

Sumário

Guia completo do usuário . . . 7
- Leia online ou como ebook . . . 7
- Percurso pedagógico . . . 7
- O modelo mental em uma frase . . . 8

Entenda o ClickUpfy . . . 9
- O problema que a ferramenta resolve . . . 9
- As quatro camadas . . . 9
- Perfis e projetos são coisas diferentes . . . 10
- Leitura compacta e leitura executável . . . 10
- Limites importantes . . . 10

Instale o ClickUpfy . . . 12
- Requisitos . . . 12
- Instale pelo npm . . . 12
- Atualize a instalação . . . 12
- Instale um binário avulso . . . 13
- Instale para desenvolvimento . . . 13
- Confirme a instalação . . . 14
- Windows: PATH e PowerShell . . . 14

Configure perfis e credenciais . . . 16
- Crie a API key . . . 16
- Automatize o setup quando necessário . . . 16
- Entenda os arquivos locais . . . 16
- Venha do ClickUpfy 0.9.0 . . . 17
- Gerencie perfis . . . 18
- Troque o workspace associado . . . 19
- Remova um perfil . . . 19

Referência do CLI: perfis, autenticação e hierarquia . . . 20
- Opções globais . . . 20
- Perfis locais . . . 25
- Workspace e identidade autenticada . . . 28
- Descoberta da hierarquia . . . 30

Prepare o primeiro projeto . . . 34
- O que você vai preparar . . . 34

Valide o perfil . . . 34
Descubra a hierarquia . . . 34
Inicialize o agente . . . 34
Confira no agente . . . 36
Libere escrita . . . 36
Primeira leitura de uma tarefa . . . 36
Navegue pela hierarquia do ClickUp . . . 38
A ordem dos recursos . . . 38
Liste workspaces e Spaces . . . 38
Liste Folders e Lists . . . 38
Confira os status da List . . . 39
Liste e busque tarefas . . . 39
Escolha o menor escopo suficiente . . . 39
Trabalhe com tarefas, subtarefas e checklists . . . 41
Leia antes de alterar . . . 41
Entenda a fila executável . . . 41
Crie uma tarefa . . . 41
Monte checklists verificáveis . . . 42
Atualize campos . . . 43
Organize por tags . . . 43
Registre progresso em comentários . . . 44
Exclua apenas com alvo confirmado . . . 44
Referência do CLI: tarefas, checklists, comentários e tempo . . . 45
Leitura de tarefas . . . 45
Criação e atualização de tarefas . . . 47
Checklists e comentários . . . 51
Time tracking . . . 55
Tags de tarefas . . . 57
Gestão do catálogo de tags . . . 58
Planeje Sprints e Sprint Points . . . 62
Como o ClickUpfy reconhece uma Sprint . . . 62
Liste os ciclos . . . 62
Encontre a Sprint atual . . . 62
Leia o relatório . . . 63
Associe uma tarefa sem mover a List principal . . . 63

Defina Sprint Points 63
Configure o MCP para Sprints 63
Referência do CLI: Sprints e Sprint Points 65
Consulta de ciclos 65
Alterações no planejamento 68
Registre tempo de trabalho 71
Consulte antes de iniciar 71
Inicie um registro 71
Encerre o registro 71
Use outro perfil em uma operação 72
Relação com comentários e status 72
Use as skills para agentes 73
O catálogo incluído 73
Inspecione antes de instalar 73
Configure o projeto 73
Instale no projeto 73
Crie uma issue sem implementá-la 74
Implemente uma issue existente 75
Use a skill diária 75
Prepare uma release 75
Leia a documentação embutida 75
Referência do CLI: Docs, skills e servidor MCP 76
Docs 76
Skills e projeto de agente 82
Conecte agentes pelo servidor MCP 89
Configure pelo terminal 89
Inicie manualmente 89
Ative read-only 90
Entenda o isolamento 90
Ferramentas de contexto e navegação 91
Use Markdown para contexto longo 93
Liste e teste o servidor 93
Diagnostique a conexão 94
Referência MCP: contexto, perfis e hierarquia 95
Ferramentas de identificação 95

Ferramentas de navegação 98
Referência MCP: Docs e administração de perfil 102
Leitura de Docs 102
Escrita de Docs 105
Administração condicionada ao servidor 108
Referência MCP: tarefas, Sprints, checklists, comentários e tempo 110
Tarefas: leitura e busca 110
Sprints 113
Ferramentas de escrita de trabalho 116
Tags de tarefas 126
Gestão do catálogo de tags 127
Consulte a referência do CLI 132
Sintaxe global 132
Saída em texto e em JSON 132
Configuração e identidade 134
Hierarquia 135
Tarefas 135
Checklists e comentários 138
Sprints 138
Time tracking 139
Docs 139
API genérica 140
Agentes e MCP 141
Manutenção e outros modos 142
Exit codes 142
Escolha o formato de saída 143
Ajuda é a referência da versão instalada 143
Resolva falhas comuns 144
O comando não foi encontrado 144
A API key foi recusada 144
O workspace está incorreto 144
Folder ou List não aparece 144
O status foi rejeitado 145
O MCP não inicia 145
O MCP recusou um ID 145

Uma ferramenta de escrita não aparece 146
A Sprint atual não foi encontrada 146
O item de checklist não mudou 146
O time entry já está em execução 146
A saída compacta não mostra um campo 146
Um script antigo quebrou depois da migração 147
Diagnóstico seguro 147

Guia completo do usuário



O ClickUpfy conecta o trabalho mantido no ClickUp ao terminal e aos agentes de código. Um único executável em Bun configura vários perfis, percorre a hierarquia do ClickUp, administra tarefas, checklists, Sprints, Docs, comentários e time tracking, instala skills no projeto e oferece as mesmas operações por MCP.

Este guia ensina essa jornada em sequência. Você começa instalando o binário e associando uma API key a um workspace. Depois encontra os IDs do projeto, executa uma tarefa completa pelo terminal e prepara um MCP restrito para que o agente trabalhe somente na List autorizada.

Leia online ou como ebook

Estas páginas também formam o **ClickUpfy, o guia completo do usuário**. A pasta do ebook publica a versão vigente em PDF e EPUB e registra os hashes que comprovam a origem comum dos formatos.

O PDF favorece leitura, compartilhamento e impressão. O EPUB se adapta ao tamanho de fonte e ao leitor digital. Os dois são reconstruídos sempre que uma página desta documentação muda.

Percurso pedagógico

Siga esta ordem na primeira leitura:

1. Entenda o ClickUpfy e os limites entre ClickUp, CLI, TUI, MCP e skills.
2. Instale o ClickUpfy pelo npm ou por um binário da GitHub Release.
3. Configure perfis e credenciais sem colocar a API key no repositório.
4. Consulte a referência de perfis e hierarquia sempre que precisar de parâmetro, efeito persistente ou variação de comando.
5. Prepare o primeiro projeto e confira o escopo antes de permitir escrita.
6. Navegue pela hierarquia até encontrar Space, Folder, List e Sprint Folder.
7. Trabalhe com tarefas e checklists, incluindo subtarefas, Markdown, datas, tags e comentários.
8. Consulte a referência de tarefas, checklists, comentários e tempo para parâmetros, formatos e confirmações de escrita.
9. Planeje Sprints e acompanhe avanço por tarefas e Sprint Points.

10. Consulte a referência de Sprints para os parâmetros de relatório, associação e Sprint Points.
11. Registre tempo no item em execução.
12. Instale e use as skills que acompanham criação, implementação e release.
13. Conecte um agente por MCP com IDs fixos e opção read-only.
14. Consulte a referência de Docs, skills e servidor MCP para os recursos de documentação dentro do ClickUp e a integração no projeto.
15. Use a referência MCP de contexto e hierarquia para conhecer cada argumento aceito pelas ferramentas de navegação.
16. Consulte a referência MCP de Docs e administração para o workspace, Docs, páginas e ferramentas condicionais do servidor.
17. Consulte a referência MCP de trabalho para operações de tarefas, Sprints, checklists, comentários e tempo.
18. Consulte a referência do CLI para comandos, argumentos e formatos de saída.
19. Resolva falhas comuns de autenticação, escopo, status e integração.

O modelo mental em uma frase

O arquivo global guarda os perfis e a API key cifrada. Os arquivos do projeto guardam somente o perfil e os IDs autorizados. O `.mcp.json`, o `.codex/config.toml`, o `.gemini/settings.json` e o `opencode.json` atendem cada cliente no seu próprio formato.

Essa divisão permite usar o mesmo ClickUpfy em vários projetos sem duplicar a API key. Um projeto pode apontar para a List de um produto, enquanto outro aponta para a List de um cliente. O MCP de cada raiz recusa IDs diferentes dos que foram fixados.

Comece por entender o papel de cada camada.

Entenda o ClickUpfy

O problema que a ferramenta resolve

O ClickUp continua sendo a fonte do trabalho: tarefa, descrição, status, checklist, comentário, Sprint e registro de tempo permanecem no workspace. O ClickUpfy não cria um gerenciador paralelo. Ele oferece uma interface previsível para consultar e alterar essa fonte pelo terminal ou por um agente.

Sem essa camada, cada agente precisa descobrir como autenticar, quais endpoints usar, em qual List trabalhar e como representar uma tarefa grande. O ClickUpfy centraliza essas decisões em três contratos:

- o perfil local associa uma API key a um workspace
- o projeto fixa o perfil e a hierarquia autorizada nos arquivos do cliente MCP
- as skills descrevem quando ler, planejar, comentar, medir tempo e concluir.

As quatro camadas

ClickUp

É o sistema remoto e a fonte de verdade. O ClickUpfy usa as APIs públicas v2 e v3 para ler workspaces, Spaces, Folders, Lists, tarefas, checklists, comentários, Sprints, Docs e time entries. Permissões e ClickApps continuam sendo administrados no ClickUp.

CLI

É a interface humana e automatizável. A sintaxe segue `recurso ação`, como `task get`, `sprint current` e `comment create`. Sem `--json`, a saída é compacta. Com `--json`, o CLI devolve um envelope estável, com `ok`, `command` e `data`, ou `ok`, `command` e `error` quando a execução falha.

TUI

É a leitura de relance. `clickupfy` sem argumentos abre a interface de terminal com a aba Overview: o perfil resolvido, o workspace, o cronômetro em execução e o estado dos arquivos locais. A TUI desta versão só mostra estado, sem telas de edição.

Servidor MCP

É a interface para agentes. Ele usa transporte stdio e expõe ferramentas com schemas explícitos. Uma configuração por projeto pode fixar account, workspace, Space, Folder, List e Sprint Folder. A List é obrigatória porque define o destino das consultas, buscas e criações de tarefa.

Skills

São instruções operacionais entregues junto ao binário. Elas não substituem o MCP: orientam o agente a usar as ferramentas na ordem adequada, preservar contexto, registrar progresso e comprovar a conclusão.

Perfis e projetos são coisas diferentes

Um **account** do ClickUpfy é um perfil local. Ele contém um nome, o usuário autenticado e o workspace associado em `~/.clickupfy/accounts.json`. A API key não fica nesse arquivo: ela é gravada cifrada no cofre `~/.clickupfy/credentials.json` como `clickup.<perfil>.api_key`.

Um **projeto** é uma raiz de código que pode conter `.mcp.json`, `.codex/config.toml`, `.gemini/settings.json` e `opencode.json`. Esses arquivos selecionam um account e fixam IDs da hierarquia no formato esperado por cada cliente. Eles podem ser versionados quando os IDs não forem considerados sensíveis pela equipe, pois nunca contêm a API key.

O mesmo account pode servir mais de um projeto. Ambientes de clientes ou workspaces diferentes devem receber perfis separados.

Leitura compacta e leitura executável

`task get` não retorna apenas a tarefa original. O ClickUpfy monta uma propriedade `execution` que achata a tarefa principal, subtarefas de qualquer nível e checklist items em uma fila ordenada.

Cada item informa:

- uma `key` estável para a leitura atual
- o tipo do item
- a relação com o pai
- a profundidade na árvore
- o estado aberto ou concluído
- a ação equivalente no CLI e no MCP.

Essa fila ajuda o agente a selecionar um item pendente sem perder a hierarquia. Quando o contexto precisa ser lido como documento, `task get --markdown` produz metadados, descrição, resumo e checkboxes em um único texto.

Limites importantes

O ClickUpfy não cria uma Sprint nem habilita o Sprint ClickApp, porque a API pública não oferece um endpoint próprio para essas operações. Crie o Sprint Folder e as Sprints na interface do ClickUp e use o CLI para consulta, associação de tarefas e Sprint Points.

O CLI também não decide sozinho qual status representa conclusão. Use `list get <list-id>` para ler os status configurados na List. Skills de implementação devem escolher um status terminal existente, não inventar uma grafia.

Tarefas, Docs e histórico não são guardados localmente. O binário persiste apenas preferências, perfis e credenciais em `~/.clickupfy/`.

Agora siga para a instalação.

Instale o ClickUpfy

Requisitos

Para usar, nenhum runtime: o ClickUpfy é um binário compilado. O pacote npm serve como launcher e exige Node.js 18.17 ou superior apenas para baixar e executar o binário da plataforma.

Para desenvolver, use Bun 1.4.4 ou superior.

Você também precisa de uma API key pessoal do ClickUp com acesso a pelo menos um workspace. A chave só será usada na etapa de configuração.

Instale pelo npm

O pacote oficial instala o comando global:

```
npm install -g @promovaweb/clickupfy
clickupfy --version
clickupfy --help
```

Na primeira execução, o launcher baixa o binário da plataforma, confere o SHA256 contra o `SHA256SUMS` da release e o mantém em cache. Uma versão válida na primeira linha confirma que o pacote e o binário foram encontrados. Se o terminal não localizar `clickupfy`, confira o diretório global do npm:

```
npm prefix --global
```

Em Linux e macOS, o diretório de executáveis costuma ser `bin` dentro do prefixo. No Windows, o prefixo contém `clickupfy.cmd`.

Atualize a instalação

O próprio comando atualiza as duas formas de instalação:

```
clickupfy upgrade
```

O alvo padrão é `latest`. Para fixar uma versão, use `clickupfy upgrade 1.0.1`. Instalado pelo npm, o comando roda `npm install -g @promovaweb/clickupfy@<alvo>`. Como binário avulso, baixa o arquivo da plataforma e o `SHA256SUMS` da GitHub Release, confere o hash e troca o executável. Rodando direto do código-fonte, o comando sai com exit code 2.

Instale um binário avulso

Cada GitHub Release publica um binário por plataforma, com o `SHA256SUMS`:

ARQUIVO	PLATAFORMA
`clickupfy-linux-x64`	Linux x64
`clickupfy-linux-arm64`	Linux ARM64
`clickupfy-darwin-x64`	macOS Intel
`clickupfy-darwin-arm64`	macOS Apple Silicon
`clickupfy-windows-x64.exe`	Windows x64

Baixe o arquivo da plataforma, confira o hash e mova para uma pasta presente no `PATH`.

No Linux ou macOS:

```
sha256sum clickupfy-linux-x64        # macOS: shasum -a 256 clickupfy-darwin-arm64
chmod +x clickupfy-linux-x64
mv clickupfy-linux-x64 ~/.local/bin/clickupfy
clickupfy --version
```

Compare o hash impresso com a linha correspondente do `SHA256SUMS`.

Antes de usar um binário baixado, confira o arquivo `SHA256SUMS` da mesma release. O nome do arquivo precisa corresponder à plataforma escolhida.

No macOS, o Gatekeeper pode impedir a abertura de um executável sem notarização. Confirme a origem no repositório oficial e siga a política de segurança da máquina. Não desative proteções globais para contornar o aviso.

Instale para desenvolvimento

Clone o repositório independente do ClickUpfy e execute:

```
bun install
bun run build
./dist/clickupfy --version
```

Para rodar sem compilar:

```
bun run dev -- --help
bun run dev -- about --json
bun run dev -- mcp --list
```

Use esse modo apenas quando precisar desenvolver ou validar uma mudança local. Para voltar ao pacote publicado, instale a versão do npm.

Confirme a instalação

Execute três leituras sem credenciais:

```
clickupfy --version
clickupfy --help
clickupfy agent skill list
```

O catálogo deve listar `clickupfy-setup`, `clickupfy-dev`, `clickup-issue-create`, `clickup-issue-implement` e `clickupfy-release`. A presença das skills confirma que os assets foram empacotados junto ao binário.

Ainda não execute comandos remotos. Primeiro configure um perfil.

Windows: PATH e PowerShell

No Windows, inclua o diretório que contém `clickupfy.cmd` no PATH do seu usuário e abra outro terminal. Para o binário avulso, extraia `clickupfy-windows-x64.exe` para uma pasta própria, adicione essa pasta ao PATH em Variáveis de Ambiente e abra outro terminal.

Se o PowerShell impedir a execução de `npm.ps1`, use `npm.cmd` e `clickupfy.cmd` explicitamente, ou execute pelo Prompt de Comando. Isso evita alterar a política de execução da máquina.

Para capturar saída JSON com acentos, prefira PowerShell 7 ou redirecionamento pelo `cmd.exe`. No PowerShell 5.1, configure UTF-8 na sessão antes de capturar:

```
[Console]::OutputEncoding = New-Object System.Text.UTF8Encoding($false)
clickupfy.cmd task list --list 30 --json | ConvertFrom-Json
```

O ClickUpfy emite UTF-8. Alterar a codepage dentro do processo filho não corrige a leitura feita pelo PowerShell pai.

As permissões `0700` e `0600` são verificadas apenas em Linux e macOS. No Windows, o `doctor` marca essas verificações como não aplicáveis; as ACLs NTFS precisam ser conferidas separadamente.

Se houver erro de escrita, feche programas que mantêm o arquivo aberto e confira o atributo somente leitura. As gravações usam substituição atômica e removem o temporário mesmo se a substituição falhar.

Configure perfis e credenciais

Crie a API key

Use uma API key pessoal criada nas configurações do ClickUp. O ClickUpfy valida a chave antes de salvar, consulta o usuário autenticado e lista os workspaces permitidos.

Não cole a chave em issue, comentário, arquivo `.env` versionado, `.mcp.json`, `.codex/config.toml` ou comando compartilhado no histórico da equipe. Em um terminal interativo, prefira o setup com prompt oculto:

```
clickupfy setup
```

Escolha um nome local para o perfil e selecione o workspace. O nome vira um identificador normalizado, como `promovaweb` ou `cliente-a`. `install` continua aceito como alias de `setup`.

A chave fica no cofre cifrado como `clickup.<perfil>.api_key`. Nenhum comando, tool ou tela mostra o valor: `status` e `account show` exibem a versão mascarada e `account list` a omite.

Automatize o setup quando necessário

Em uma automação isolada, o setup aceita parâmetros:

```
clickupfy setup \
  --api-key "$CLICKUP_API_KEY" \
  --name "promovaweb" \
  --workspace "9007154145" \
  --non-interactive
```

Evite colocar a chave diretamente em um script. Leia o valor de um secret manager ou de uma variável protegida e apague o ambiente temporário depois da execução. `--token` continua aceito como alias de `--api-key`.

Entenda os arquivos locais

Por padrão, os arquivos ficam em `~/.clickupfy/`:

ARQUIVO	CONTEÚDO	PERMISSÃO
`accounts.json`	perfis, sem segredos	0600
`credentials.json`	envelope AES-256-GCM com as API keys	0600
`master.key`	master key em hexadecimal	0600

ARQUIVO	CONTEÚDO	PERMISSÃO
`settings.json`	preferências em texto claro	0600

A pasta é criada com `0700`. O `accounts.json` permite vários perfis:

```
{
  "version": 1,
  "activeAccount": "promovaweb",
  "accounts": {
    "promovaweb": {
      "name": "promovaweb",
      "user": {
        "id": 123,
        "username": "Desenvolvedor",
        "email": "dev@example.com"
      },
      "workspace": {
        "id": "456",
        "name": "Engenharia"
      },
      "createdAt": "2026-11-13T12:00:00.000Z",
      "updatedAt": "2026-11-13T12:00:00.000Z"
    }
  }
}
```

Esse arquivo pode ser lido e copiado sem expor a chave. A API key de cada perfil fica no cofre e exige a master key, que vem de `CLICKUPFY_MASTER_KEY` ou de `~/.clickupfy/master.key`.

Venha do ClickUpfy 0.9.0

Na primeira execução de um comando que lê perfis, quando o `accounts.json` ainda não existe e o config antigo existe (`PROMOVAWEB_CLICKUPFY_CONFIG` ou `~/clickupfy/config.json`), o binário importa cada perfil: os metadados vão para `accounts.json` e cada API key para o cofre.

O arquivo antigo só é lido. Ele não é alterado nem apagado, então o ClickUpfy 0.9.0 continua funcionando durante a transição. O stderr mostra `Imported N profile(s) from <caminho> into the vault.` uma única vez.

Para repetir a importação sob demanda:

```
clickupfy account import
clickupfy account import --force
```

Sem `--force`, perfis que já existem são pulados e aparecem em `skipped`; com `--force`, eles são substituídos pelos dados do arquivo antigo. Em JSON, o comando devolve `{ source, imported, skipped, activeAccount }`, sem chaves.

Gerencie perfis

Use comandos que mascaram a API key:

```
clickupfy account list
clickupfy account show
clickupfy status
clickupfy whoami
```

`status` mostra o caminho do `accounts.json`, o perfil resolvido, se ele é o ativo, a chave mascarada, o usuário e o workspace. Não chama a API. `whoami` faz uma chamada autenticada e confirma se a chave ainda é válida.

Para conferir o setup local, as permissões dos arquivos, o schema, o cofre e os arquivos de integração do projeto, execute:

```
clickupfy doctor
```

Esse diagnóstico não faz uma chamada ao ClickUp. Use `clickupfy doctor --online` para validar a API key remotamente e ler as skills instaladas com `npx skills list`.

Para trocar o perfil ativo:

```
clickupfy account use cliente-a
```

Para usar outro perfil em um único comando, sem alterar o ativo:

```
clickupfy --account cliente-a task get 86abc123
```

A variável `PROMOVAWEB_CLICKUPFY_ACCOUNT` oferece a mesma seleção em automações. A ordem de resolução é `--account`, a variável e o perfil ativo. A variável `PROMOVAWEB_CLICKUPFY_CONFIG` aponta para outro config antigo e é útil em testes ou ambientes efêmeros.

Troque o workspace associado

Liste os workspaces autorizados e escolha um deles:

```
clickupfy workspace list
clickupfy workspace use 9007154145
```

Essa ação altera o workspace do perfil resolvido. Configurações de projeto que fixam `--workspace` precisam continuar correspondendo ao perfil.

Remova um perfil

```
clickupfy account remove cliente-antigo
```

O comando pede confirmação e apaga os metadados e a credencial do perfil. Em automação, `--yes` confirma sem prompt. A remoção apaga o perfil local, não revoga a API key no ClickUp. Revogue a chave no ClickUp quando ela não for mais necessária.

Com o perfil validado, prepare o primeiro projeto.

Referência do CLI: perfis, autenticação e hierarquia

Opções globais

Estas opções valem em qualquer posição, antes ou depois do comando:

OPÇÃO	TIPO	EFEITO
`-h`, `--help`	booleano	Help do comando atual, com exit code `0`.
`-V`, `--version`	booleano	Imprime `clickupfy <versão>`, com exit code `0`.
`--json`	booleano	Saída em JSON com o envelope `{ ok, command, data }`.
`--skill [topic]`	texto	Documentação embutida. Topics: `cli`, `tui`, `mcp`, `config`, `exit-codes` e `reference`.
`--opencli`	booleano	Descrição OpenCLI da árvore de comandos em JSON.
`--account <perfil>`	texto	Seleciona um perfil para esta execução sem trocar o perfil ativo.
`--no-color`	booleano	Desliga cores ANSI. `NO_COLOR=1` tem o mesmo efeito.

O perfil usado por um comando é resolvido nesta ordem: a opção `--account <perfil>`, a variável `PROMOVAWEB_CLICKUPFY_ACCOUNT` e o perfil ativo em `accounts.json`. Sem nenhum deles, o comando sai com o exit code `4` e sugere `clickupfy setup`. Um perfil inexistente sai com `6`. O perfil ativo só muda com `account use`.

Como as opções globais valem em qualquer posição, as duas formas abaixo são equivalentes:

```
clickupfy --account cliente-a task list
clickupfy task list --account cliente-a
```

O envelope do `--json`

Com `--json`, a saída de sucesso é um objeto com `ok`, `command` e `data`:

```
{
  "ok": true,
  "command": "clickupfy status",
  "data": { "account": "produto", "active": true }
}
```

Em erro, o objeto traz `ok: false`, `command` e `error` com `code`, `name`, `message` e `details`:

```
{
  "ok": false,
  "command": "clickupfy account show",
  "error": {
    "code": 6,
    "name": "NOT_FOUND",
    "message": "Profile \"cliente-x\" does not exist.",
    "details": { "account": "cliente-x" }
  }
}
```

O exit code do processo é sempre o mesmo de `error.code`. Um script deve ler os dois: o envelope para a mensagem e o code para a classe da falha. Listas vêm dentro de um objeto, como `{ "tasks": [] }` ou `{ "spaces": [] }`, e nunca como array na raiz de `data`.

`clickupfy upgrade`

Atualiza o ClickUpfy para a versão mais recente ou para uma versão indicada. Instalado pelo pacote npm, roda `npm install -g @promovaweb/clickupfy@<alvo>`. Como binário avulso, baixa `clickupfy-<plataforma>` e o `SHA256SUMS` da GitHub Release, confere o hash e troca o executável.

PARÂMETRO	OBRIGATÓRIO	USO
`[alvo]`	não	`latest`, `next` (só npm) ou uma versão SemVer, como `1.0.1`. `latest` é o padrão.

Rodando do código-fonte (`bun run src/main.ts`), o comando se recusa a atualizar e sai com `2`. Alvo inválido sai com `2`; asset ausente para a plataforma, com `6`; falha de rede ou de checksum, com `5`; falta de permissão para trocar o arquivo, com `7`.

Exemplos:

```
clickupfy upgrade
```

```
clickupfy upgrade latest
```

```
clickupfy upgrade next
```

```
clickupfy upgrade 1.0.1
```

```
clickupfy upgrade v1.0.1
```

O quinto exemplo normaliza o prefixo `v` antes de montar a URL da release. O canal `next` existe apenas no npm: como binário avulso, use `latest` ou uma versão.

`clickupfy doctor`

Confere o ambiente local sem alterar arquivos. O diagnóstico valida Bun 1.4.4 ou superior, o TTY de entrada e saída, a pasta de dados com `0700`, a leitura de `settings.json`, a origem da master key, a decifragem do cofre, a versão e a plataforma. Em seguida verifica os perfis e os arquivos MCP do projeto atual.

CHECK	O QUE CONFERE
`accounts`	`~/.clickupfy/accounts.json` existe e segue o schema.
`profile`	O perfil resolvido por `--account`, `PROMOVAWEB_CLICKUPFY_ACCOUNT` ou o perfil ativo.
`api-key`	A API key do perfil está no cofre, sem mostrar o valor.
`legacy-config`	O config do ClickUpfy 0.9.0 ainda existe e guarda API keys em texto puro.
`mcp-json`, `codex-config`	Servidores `clickupfy` no `.mcp.json` e no `.codex/config.toml` do diretório atual.

Cada linha usa `✓`, `!` ou `✗`. Em JSON, `data` traz `{ checks, ok }`, e cada check tem `name`, `status` (`ok`, `warn` ou `fail`), `detail` e, quando se refere a um arquivo, `path`. Arquivo ainda não criado e ausência de TTY geram `warn`. Um check com `fail` produz exit code `3`; os demais retornam `0`.

Com `--online`, o doctor também valida a API key em `GET /user` e lê as skills instaladas com `npx skills list --json`. Essas verificações chamam a rede e podem baixar o gerenciador de skills.

Exemplos:

```
clickupfy doctor
```

```
clickupfy doctor --json
```

```
clickupfy doctor --online
```

```
cd ~/projetos/produto && clickupfy doctor
```

```
clickupfy doctor --json | jq '.data.checks[] | select(.status == "fail")'
```

O diagnóstico confirma a configuração local e os arquivos usados pelos agentes, mas sem `--online` não testa se a API key continua válida no ClickUp. Para essa verificação remota, use `clickupfy whoami`.

`clickupfy setup`

Cria ou atualiza um perfil local e associa a API key a um workspace autorizado. O comando valida a chave em `GET /user`, lista os workspaces que ela alcança e grava a chave cifrada no cofre. `clickupfy install` é um alias compatível.

Sem `--non-interactive`, o comando pergunta pelos valores ausentes, inclusive a chave sem eco. Fora de um terminal interativo, `--api-key` é obrigatória e `--workspace` também quando a chave alcança mais de um workspace. Repetir o comando com o mesmo `--name` substitui aquele perfil.

PARÂMETRO	OBRIGATÓRIO	USO
`--api-key <chave>`	condicional	API key pessoal do ClickUp (`pk_...`). `--token` é um alias compatível.
`--name <nome>`	não	Nome legível do perfil, normalizado para kebab-case. O padrão é o username do ClickUp.
`--workspace <id>`	condicional	Workspace associado ao perfil. Obrigatório fora de um TTY quando a chave alcança vários.
`--non-interactive`	não	Recusa prompts e sai com `2` quando falta um valor.

Exemplos:

```
clickupfy setup
```

```
clickupfy setup --name produto
```

```
clickupfy setup --name cliente-a --workspace 90070363783
```

```
clickupfy setup --api-key "$CLICKUP_API_KEY" --name automacao
```

```
clickupfy setup \
  --api-key "$CLICKUP_API_KEY" \
  --name produto \
  --workspace 90070363783 \
  --non-interactive \
  --json
```

Quando a API key for inválida, execute `whoami` para conferir a autenticação. Quando o workspace informado não pertencer à chave, o comando sai com `6` e lista os IDs disponíveis em `error.details.available`. Nunca escreva a chave diretamente no histórico do shell; prefira uma variável de ambiente temporária ou a entrada interativa.

`clickupfy status`

Mostra o perfil resolvido, o caminho do `accounts.json`, se ele é o ativo, a chave mascarada, o usuário e o workspace associado. O comando não chama a API do ClickUp.

Em JSON, `data` traz `{ config, account, active, name, apiKey, user, workspace }`. Use este comando como inspeção local, não como teste definitivo de uma chave recém-revogada.

Exemplos:

```
clickupfy status
```

```
clickupfy status --json
```

```
clickupfy status --account produto
```

```
clickupfy --account cliente-a status --json
```

```
clickupfy status > /tmp/clickupfy-status.txt
```

O terceiro exemplo consulta o perfil `produto` sem alterar o perfil que será usado pela próxima chamada sem `--account`. A redireção do último exemplo é segura porque a saída mascara a chave, mas ainda pode conter nomes de pessoas e de workspaces.

Perfis locais

Um perfil é um conjunto local de API key, identidade autenticada e workspace selecionado. Cada perfil tem um identificador, como `produto` ou `cliente-a`. O identificador é usado com `--account` no CLI e pelo `--account` do servidor MCP.

Os perfis ficam em `~/.clickupfy/accounts.json`, com `version`, `activeAccount` e o mapa `accounts`. Esse arquivo não guarda segredo: a API key de cada perfil fica no cofre. Por isso ele pode ser lido e copiado sem expor a chave.

`clickupfy account list`

Lista todos os perfis e marca o ativo com `*`. Cada linha traz `id`, `active`, `name`, `user`, `workspace` e `workspace_id`. Nenhuma API key aparece. `clickupfy account ls` é o alias equivalente.

Exemplos:

```
clickupfy account list
```

```
clickupfy account ls
```

```
clickupfy account list --json
```

```
clickupfy account list --account produto
```

```
clickupfy account list | less
```

Mesmo com `--account`, a lista continua mostrando todos os perfis; a opção só tem efeito quando uma ação precisa resolver uma conta. Em JSON, `data` traz `{ accounts: [] }`. Use a tabela para conferir o identificador exato antes de chamar `account show`, `account use` ou um comando de trabalho.

`clickupfy account show [perfil]`

Exibe metadados seguros de um perfil, com a API key mascarada. Sem argumento, resolve o perfil ativo ou o que foi indicado pela opção global `--account`.

PARÂMETRO	OBRIGATÓRIO	USO
`[perfil]`	não	Identificador local do perfil.

Exemplos:

```
clickupfy account show
```

```
clickupfy account show produto
```

```
clickupfy account show --account cliente-a
```

```
clickupfy account show produto --json
```

```
clickupfy account show produto --account cliente-a
```

No último caso, a saída é do perfil `produto`, pois o argumento posicional tem precedência sobre a opção global. Se o perfil não existir, o comando sai com `6` sem iniciar uma requisição ao ClickUp.

`clickupfy account use <perfil>`

Define o perfil ativo em `accounts.json`. É uma alteração local e não muda nada no ClickUp. Evite usar este comando em automações ou quando dois projetos usam contas diferentes ao mesmo tempo; nesses casos, informe `--account` em cada chamada ou fixe o perfil no MCP de cada projeto.

PARÂMETRO	OBRIGATÓRIO	USO
`<perfil>`	sim	Identificador local já configurado.

Exemplos:

```
clickupfy account use produto
```

```
clickupfy account use cliente-a
```

```
clickupfy account use homologacao
```

```
clickupfy account use suporte
```

```
clickupfy account use produto && clickupfy status
```

O último exemplo confirma a mudança persistida. Em JSON, `data` traz `{ activeAccount }`. Quando o comando faz parte de um script, prefira não depender do perfil ativo: `clickupfy --account produto ...` deixa o destino explícito e não interfere no restante da máquina.

`clickupfy account remove <perfil>`

Remove os metadados do perfil e a API key correspondente do cofre. Não revoga a API key no ClickUp, não remove usuários e não exclui tarefas. Em terminal interativo, pede confirmação; fora dele, `--yes` é obrigatório.

PARÂMETRO	OBRIGATÓRIO	USO
`<perfil>`	sim	Identificador local a remover.
`--yes`	não	Confirma sem interação.

Exemplos:

```
clickupfy account remove conta-antiga
```

```
clickupfy account remove sandbox --yes
```

```
clickupfy account remove cliente-encerrado --yes
```

```
clickupfy account remove homologacao --yes
```

```
clickupfy account remove temporario --yes && clickupfy account list
```

Recusar a confirmação sai com `130`. Se você remover o perfil ativo, o ClickUpfy torna ativo o primeiro perfil que sobrar ou deixa `activeAccount` como `null` quando não houver outro. Leia `account list` após a remoção, especialmente em uma máquina compartilhada.

`clickupfy account import`

Importa os perfis do config do ClickUpfy 0.9.0 para o cofre cifrado. O comando lê `PROMOVAWEB_CLICKUPFY_CONFIG` ou `~/clickupfy/config.json`, grava os metadados em `accounts.json` e cada API key como `clickup.<perfil>.api_key`. O arquivo antigo só é lido: ele não é alterado nem apagado, então o ClickUpfy 0.9.0 continua funcionando durante a transição.

A mesma importação roda automaticamente na primeira execução de um comando que lê perfis, quando `accounts.json` ainda não existe e o config antigo está no lugar. O stderr mostra `Imported N profile(s) from <caminho> into the vault.` uma única vez.

PARÂMETRO	OBRIGATÓRIO	USO
`--force`	não	Substitui perfis que já existem em vez de pulá-los.

Exemplos:

```
clickupfy account import
```

```
clickupfy account import --force
```

```
clickupfy account import --json
```

```
PROMOVAWEB_CLICKUPFY_CONFIG=/tmp/clickupfy-config.json clickupfy account import
```

```
clickupfy account import --force && clickupfy account list
```

Sem `--force`, os perfis que já existem são pulados e aparecem em `skipped`. Em JSON, `data` traz `{ source, imported[], skipped[], activeAccount }`, sem chaves. Config antigo inválido faz o comando sair com `3` e informar o caminho.

Workspace e identidade autenticada

O perfil tem um workspace associado, mas uma API key pode ter permissão para mais de um workspace. `workspace list` descobre as opções; `workspace use` persiste a escolha no perfil; `whoami` consulta a API e confirma o usuário.

`clickupfy workspace list`

Lista os workspaces que a API key do perfil consegue acessar. O marcador `*` indica o workspace atualmente associado ao perfil. `workspace ls` é o alias.

Exemplos:

```
clickupfy workspace list
```

```
clickupfy workspace ls
```

```
clickupfy workspace list --account produto
```

```
clickupfy --account cliente-a workspace list --json
```

```
clickupfy workspace list | tee /tmp/workspaces.txt
```

Em JSON, `data` traz `{ workspaces: [] }`. O resultado autoritativo vem da API. Caso o workspace esperado não apareça, verifique a permissão da API key no ClickUp em vez de tentar adivinhar um ID.

`clickupfy workspace use <workspace-id>`

Associa um workspace autorizado ao perfil resolvido. O comando consulta a API e só salva o ID quando ele aparece na lista permitida, portanto um número copiado de outro perfil é recusado com `6`.

PARÂMETRO	OBRIGATÓRIO	USO
`<workspace-id>`	sim	ID devolvido por `workspace list`.

Exemplos:

```
clickupfy workspace use 90070363783
```

```
clickupfy workspace use 9007154145 --account cliente-a
```

```
clickupfy --account produto workspace use 90070363783
```

```
clickupfy workspace use 9007154145 && clickupfy status
```

```
clickupfy --account suporte workspace use 90070363783
```

A alteração afeta somente o perfil indicado. Um servidor MCP com `--workspace` fixo ainda recusa um workspace diferente, mesmo que o perfil local seja alterado depois.

`clickupfy whoami`

Valida a API key em uma chamada ao ClickUp e devolve o usuário autenticado junto do perfil e do workspace resolvidos. Em JSON, `data` traz `{ account, user }`. Use o comando depois de `setup`, após uma troca de chave ou ao investigar erro de autorização.

Exemplos:

```
clickupfy whoami
```

```
clickupfy whoami --json
```

```
clickupfy whoami --account produto
```

```
clickupfy --account cliente-a whoami --json
```

```
clickupfy whoami && clickupfy workspace list
```

Uma resposta de `status` não substitui este teste: ela lê a configuração local. `whoami` é a chamada que confirma se a chave ainda funciona no serviço remoto.

Descoberta da hierarquia

O ClickUp organiza trabalho em workspace, Space, Folder e List. Uma List pode existir diretamente dentro de um Space, sem Folder. IDs não são intercambiáveis: um `space-id` não serve no lugar de um `folder-id`, por exemplo. Comece no workspace selecionado e preserve os IDs retornados em um local seguro.

Em texto, as listagens saem como tabela. Com `--json`, cada uma vem dentro de um objeto, como `{ "spaces": [] }`, `{ "folders": [] }` ou `{ "lists": [] }`.

`clickupfy space list`

Lista Spaces do workspace associado ao perfil. `--archived` acrescenta Spaces arquivados; sem a opção, o resultado contém apenas os ativos.

PARÂMETRO	OBRIGATÓRIO	USO
`--archived`	não	Inclui Spaces arquivados.

Exemplos:

```
clickupfy space list
```

```
clickupfy space ls
```

```
clickupfy space list --archived
```

```
clickupfy space list --account produto --json
```

```
clickupfy --account cliente-a space list --archived
```

Escolha o Space pelo ID e pelo nome retornados, não pela posição da linha. Spaces arquivados podem ser úteis em migração ou consulta histórica, mas não devem ser usados como destino de novas tarefas sem autorização explícita.

`clickupfy folder list --space <id>`

Lista Folders de um Space. O parâmetro `--space` é obrigatório no CLI; o comando não usa um Space implícito porque um perfil pode operar em vários projetos.

PARÂMETRO	OBRIGATÓRIO	USO
`--space <id>`	sim	ID do Space retornado por `space list`.
`--archived`	não	Inclui Folders arquivados.

Exemplos:

```
clickupfy folder list --space 90070363783
```

```
clickupfy folder ls --space 90070363783
```

```
clickupfy folder list --space 90070363783 --archived
```

```
clickupfy folder list --space 90070363783 --account produto --json
```

```
clickupfy --account cliente-a folder list --space 9007154145 --archived
```

Uma resposta vazia não significa necessariamente falta de acesso: o Space pode guardar Lists diretamente. Execute `list list --space <id>` para cobrir esse ramo da hierarquia.

`clickupfy list list --folder <id> | --space <id>`

Lista Lists dentro de um Folder ou Lists que pertencem diretamente a um Space. Informe uma das duas opções. Quando as duas aparecem, `--folder` tem precedência e o Space não é enviado à API. Sem nenhuma delas, o comando sai com `2`.

PARÂMETRO	OBRIGATÓRIO	USO
`--folder <id>`	condicional	ID do Folder que contém as Lists.
`--space <id>`	condicional	ID do Space para Lists sem Folder.
`--archived`	não	Inclui Lists arquivadas.

Exemplos:

```
clickupfy list list --folder 2001
```

```
clickupfy list ls --folder 2001
```

```
clickupfy list list --folder 2001 --archived
```

```
clickupfy list list --space 90070363783
```

```
clickupfy --account produto list list --space 90070363783 --archived --json
```

Consulte cada List pelo comando seguinte para obter os status configurados. Em projetos com MCP, o ID da List escolhido aqui será o valor obrigatório de `agent install --list` ou de `mcp serve --list`.

`clickupfy list get <list-id>`

Obtém a List e os status aceitos pelas tarefas dela. A grafia retornada para um status é a mesma que deve aparecer em `task create --status` e `task update --status`; não use uma tradução ou suposição local.

PARÂMETRO	OBRIGATÓRIO	USO
`<list-id>`	sim	ID da List retornado por `list list`.

Exemplos:

```
clickupfy list get 901328052926
```

```
clickupfy list get 901328052926 --json
```

```
clickupfy list get 901328052926 --account produto
```

```
clickupfy --account cliente-a list get 901328060001 --json
```

```
clickupfy list get 901328052926 --json > /tmp/list-901328052926.json
```

O `data` do `--json` traz a List da API com `statuses[]`. O último exemplo guarda o objeto para outro programa; a tabela compacta é adequada para leitura, mas não é uma API estável de máquina. Continue em tarefas e checklists para usar os status e IDs encontrados aqui.

Prepare o primeiro projeto

O que você vai preparar

Neste percurso, um repositório será ligado a uma List do ClickUp. O resultado são `.mcp.json` e `.codex/config.toml`, cada um no formato do cliente que os consome, iniciando o ClickUpfy com um perfil e IDs fixos, além das cinco skills instaladas no projeto.

Antes de escrever qualquer tarefa, você vai:

1. validar o perfil
2. descobrir Space, Folder e List
3. instalar o MCP em read-only para conferir o destino
4. liberar escrita somente depois da revisão.

Valide o perfil

```
clickupfy status
clickupfy whoami
```

Confirme o nome do perfil, o usuário e o workspace. Se qualquer dado estiver errado, volte à configuração.

Descubra a hierarquia

```
clickupfy workspace list
clickupfy space list
clickupfy folder list --space <space-id>
clickupfy list list --folder <folder-id>
```

Uma List pode pertencer diretamente ao Space. Nesse caso:

```
clickupfy list list --space <space-id>
```

Guarde os IDs confirmados. Se o projeto usa Sprints, localize também o Sprint Folder que contém as Lists temporais.

Inicialize o agente

Na raiz do repositório:

```
clickupfy --account promovaweb agent install \
  --workspace 123 \
  --space 10 \
  --folder 20 \
  --list 30
```

`space` e `list` são obrigatórios. Workspace e Folder podem ser omitidos quando não forem necessários ao escopo. Acrescente `--sprint-folder <id>` se o projeto usa Sprints. `agent init` é o alias de `agent install`.

O comando instala as skills e mescla o servidor no `.mcp.json` e no `.codex/config.toml` existentes sem remover outros servidores ou configurações. No JSON, a entrada fica assim:

```
{
  "mcpServers": {
    "promovaweb-clickupfy": {
      "command": "clickupfy",
      "args": [
        "mcp",
        "serve",
        "--account",
        "promovaweb",
        "--workspace",
        "123",
        "--space",
        "10",
        "--folder",
        "20",
        "--list",
        "30"
      ]
    }
  }
}
```

No Codex, a entrada equivalente fica na tabela `mcp_servers`:

```
[mcp_servers."promovaweb-clickupfy"]
command = "clickupfy"
args = ["mcp", "serve", "--account", "promovaweb", "--workspace", "123", "--space",
"10", "--folder", "20", "--list", "30"]
```

Só o id do perfil vai para os arquivos, nunca a API key.

Se o projeto usa outros clientes, `clickupfy mcp setup` grava a mesma entrada em `.mcp.json`, `.codex/config.toml`, `.gemini/settings.json` e `opencode.json`:

```
clickupfy mcp setup \
  --name promovaweb-clickupfy \
  --account promovaweb \
  --space 10 \
  --list 30 \
  --clients claude-code,codex,gemini,opencode \
  --non-interactive
```

Antes de abrir o cliente MCP pela primeira vez, acrescente `--read-only` ao fim de `args`. O `agent install` prepara o servidor completo. Essa edição consciente reduz a superfície durante a conferência inicial.

Confira no agente

Reinicie ou recarregue o cliente MCP para que ele leia a nova configuração. Peça uma consulta ao contexto:

```
Use clickupfy_mcp_context e mostre o perfil, o workspace e a hierarquia deste
projeto.
```

Compare o resultado com os IDs anotados. Depois liste as tarefas:

```
Use clickupfy_tasks_list sem informar listId.
```

O servidor deve aplicar a List configurada. Uma tentativa de consultar outra List deve ser recusada.

Libere escrita

Quando o destino estiver correto, remova apenas `--read-only` da entrada gerenciada. Reabra o cliente MCP e confirme que as ferramentas de escrita aparecem em `tools/list`.

Crie uma tarefa de teste somente se o projeto e a equipe autorizarem a mutação. Caso contrário, use uma tarefa real já existente para validar leitura, comentários e checklists.

Primeira leitura de uma tarefa

No terminal:

```
clickupfy task get <task-id> --markdown
```

No agente:

```
Leia <task-id> com clickupfy_task_get usando markdown: true. Não altere a
tarefa.
```

Confira descrição, subtarefas, checklists e fila executável. Agora você já pode aprofundar a hierarquia ou começar a trabalhar com tarefas.

Navegue pela hierarquia do ClickUp

A ordem dos recursos

O ClickUp organiza o trabalho nesta cadeia:

```
Workspace
└── Space
    ├── List sem Folder
    └── Folder
        └── List
```

Sprints também são Lists, mas vivem normalmente dentro de um Sprint Folder e possuem `start_date` e `due_date`.

O perfil já conhece um workspace. Por isso, `space list` não exige esse ID. A partir do Space, os comandos precisam do pai para evitar uma busca ambígua.

Liste workspaces e Spaces

```
clickupfy workspace list
clickupfy space list
clickupfy space list --archived
```

O asterisco na saída compacta indica o workspace associado ao perfil. Use `--archived` somente quando precisar localizar uma estrutura antiga.

Liste Folders e Lists

```
clickupfy folder list --space <space-id>
clickupfy list list --folder <folder-id>
```

Para Lists que pertencem diretamente ao Space:

```
clickupfy list list --space <space-id>
```

Folder e Space são alternativas no comando de Lists. Não informe os dois ao mesmo tempo: sem nenhum deles o comando sai com exit code 2.

Confira os status da List

```
clickupfy list get <list-id>
```

Essa leitura é importante antes de atualizar uma tarefa. Os nomes e tipos de status são configuráveis no ClickUp. Uma equipe pode usar `feito`, outra `concluído`, e uma terceira pode manter mais de um estado terminal.

Registre o ID da List e escolha um status terminal existente. Não assuma que `done` ou `complete` será aceito.

Liste e busque tarefas

```
clickupfy task list --list <list-id>
clickupfy task search --query "autenticação"
```

`task list` consulta uma List conhecida. `task search` procura no workspace, mas o MCP de projeto restringe a busca à List fixada para impedir vazamento de contexto entre projetos. Vários valores de `--status` e `--assignee` são separados por vírgula:

```
clickupfy task list --list <list-id> --status "to do,in progress"
clickupfy task list --list <list-id> --assignee 7,8
```

Use `--json` quando precisar inspecionar campos não exibidos na tabela:

```
clickupfy task list --list <list-id> --json
```

A resposta segue o envelope `{ ok, command, data }`, e as tarefas ficam em `data.tasks`. Options globais valem em qualquer posição, antes ou depois do comando.

Escolha o menor escopo suficiente

Um MCP de projeto sempre exige List. Space e Folder ajudam as ferramentas de navegação, mas não ampliam o destino de criação. Sprint Folder só deve ser configurado quando as Sprints do projeto estiverem dentro dele.

Para um projeto com List diretamente no Space:

```
clickupfy --account produto agent install \
  --space 10 \
  --list 30
```

Para um projeto com Folder e Sprints:

```
clickupfy --account produto agent install \
  --space 10 \
  --folder 20 \
  --list 30 \
  --sprint-folder 40
```

Continue em tarefas e checklists.

Trabalhe com tarefas, subtarefas e checklists

Leia antes de alterar

Comece pela tarefa completa:

```
clickupfy task get <task-id> --markdown
```

Esse formato reúne metadados, descrição, subtarefas e checklist items em um documento. Para automação estruturada, use:

```
clickupfy task get <task-id> --json
```

Para obter somente a resposta original da API, sem a fila `execution`:

```
clickupfy task get <task-id> --raw
```

`--raw` e `--markdown` são mutuamente exclusivos. Em `--json`, os itens ficam em `data.task` e `data.execution`.

Entenda a fila executável

O objeto `execution` transforma a árvore em itens ordenados. Uma chave como `task:86abc123` identifica uma tarefa ou subtarefa. Uma chave como `checklist:check-1:item-1` identifica um item de checklist.

O resumo informa totais abertos e concluídos. Cada item inclui `parentKey` e `depth`, por isso um agente consegue trabalhar em uma subtarefa aninhada sem tratá-la como item independente. O campo `action` traz o comando do CLI e a chamada MCP que concluem o item.

A fila é uma projeção da leitura atual, não um segundo estado. Sempre releia a tarefa depois de uma mutação.

Crie uma tarefa

Use Markdown na descrição quando o ClickUp ClickApp correspondente estiver disponível:

```
clickupfy task create \
  --list <list-id> \
  --name "Implementar autenticação" \
  --markdown-content "Adicionar o fluxo de login e os testes." \
  --start-date 2026-08-01 \
  --due-date 2026-08-05
```

Datas aceitam o formato `AAAA-MM-DD` e são gravadas no início do dia no fuso local da máquina. Antes de criar, confira o fuso e a política de datas da equipe.

Para criar uma subtarefa, informe o pai:

```
clickupfy task create \
  --list <list-id> \
  --name "Criar testes de autenticação" \
  --parent <task-id> \
  --markdown-content "Cobrir sucesso, credencial inválida e limite."
```

Subtarefas podem receber novas subtarefas. A leitura executável preserva todos os níveis. A tarefa criada volta com `id` e `url`.

Monte checklists verificáveis

Crie o checklist na tarefa e depois os itens:

```
clickupfy checklist create <task-id> --name "Validação"
clickupfy checklist item-create <checklist-id> --name "Executar testes unitários"
clickupfy checklist item-create <checklist-id> --name "Executar build"
```

Os itens são criados abertos. Marque um item somente depois de executar sua verificação:

```
clickupfy checklist set \
  <task-id> <checklist-id> <item-id> \
  --resolved
```

Para reabrir:

```
clickupfy checklist set \
  <task-id> <checklist-id> <item-id> \
  --open
```

O ClickUpfy confirma primeiro que o item pertence à tarefa, grava o novo estado e relê a tarefa. O comando só comunica sucesso quando a API devolve o valor esperado.

Atualize campos

```
clickupfy task update <task-id> --status "in progress"
clickupfy task update <task-id> --priority 2
clickupfy task update <task-id> --start-date 2026-08-01
clickupfy task update <task-id> --due-date 2026-08-05
clickupfy task update <task-id> --points 5
```

As prioridades seguem a API do ClickUp:

VALOR	PRIORIDADE
`1`	urgente
`2`	alta
`3`	normal
`4`	baixa

Consulte `list get` antes de definir status. Para remover datas:

```
clickupfy task update <task-id> --clear-start-date
clickupfy task update <task-id> --clear-due-date
```

`task update` exige ao menos um campo.

Organize por tags

```
clickupfy tag list --space <space-id>
clickupfy task add-tag <task-id> backend
clickupfy task remove-tag <task-id> backend
```

`task add-tag` e `task remove-tag` preservam as outras tags da tarefa. Use `--custom-task-id` quando o argumento for um id personalizado do workspace.

Registre progresso em comentários

```
clickupfy comment list --task <task-id>
clickupfy comment create \
  --task <task-id> \
  --text "Testes focais passaram. Iniciando a regressão."
```

Um comentário útil registra um evento verificável: início, impedimento, checkpoint de teste ou conclusão. Evite publicar raciocínio interno, secrets, logs extensos ou promessas sem evidência. `--notify-all` notifica os participantes da tarefa.

Exclua apenas com alvo confirmado

```
clickupfy task delete <task-id> --yes
```

A exclusão é permanente na interface do CLI e exige confirmação. Leia a tarefa, confira List e nome e confirme se a solicitação autoriza exclusão antes de usar `--yes`. Fora de um terminal interativo, sem `--yes`, o comando sai com exit code 2; recusar a confirmação sai com 130.

Quando a equipe trabalha por ciclos, continue em Sprints e Sprint Points.

Referência do CLI: tarefas, checklists, comentários e tempo

Leitura de tarefas

`clickupfy task list --list <id>`

Lista uma página de até 100 tarefas de uma List. `--list` é obrigatório porque o CLI não lê o `.mcp.json` do projeto. `--status` aceita um ou mais nomes de status, `--assignee` recebe um ou mais IDs de responsáveis e `--page` começa em zero.

PARÂMETRO	OBRIGATÓRIO	USO
`--list <id>`	sim	List consultada.
`--status <valores>`	não	Restringe aos status informados, separados por vírgula.
`--assignee <valores>`	não	Restringe aos responsáveis, separados por vírgula.
`--include-closed`	não	Inclui tarefas concluídas.
`--page <n>`	não	Página, iniciando em `0`.

Exemplos:

```
clickupfy task list --list 901328052926
```

```
clickupfy task list --list 901328052926 --status "in progress"
```

```
clickupfy task list --list 901328052926 --status "to do,in progress" --assignee 42
```

```
clickupfy task list --list 901328052926 --include-closed --page 0
```

```
clickupfy task list --list 901328052926 --page 2 --account produto --json
```

A saída comum é uma tabela com ID, nome, status, prioridade, pontos, pessoas e link. Com `--json`, `data` traz `{ tasks: [] }` e o CLI devolve as tarefas completas da API. Para encontrar uma tarefa fora de uma List conhecida, use `task search`.

`clickupfy task get <task-id>`

Obtém uma tarefa e monta uma fila chamada `execution`, que inclui a tarefa, suas subtarefas e seus checklist items em uma ordem estável. Sem opção de formato, imprime uma tabela compacta da fila. Com `--json`, `data` traz `{ task, execution: { summary, items[] } }`. `--markdown` junta metadados, descrição e itens em um documento; `--raw` devolve apenas a resposta original da API.

PARÂMETRO	OBRIGATÓRIO	USO
`<task-id>`	sim	Tarefa raiz a consultar.
`--raw`	não	Retorna somente a resposta original do ClickUp.
`--markdown`	não	Renderiza descrição e fila em Markdown.

Exemplos:

```
clickupfy task get 86abc123
```

```
clickupfy task get 86abc123 --markdown
```

```
clickupfy task get 86abc123 --raw
```

```
clickupfy task get 86abc123 --json
```

```
clickupfy task get 86abc123 --markdown --account produto
```

Cada item de `execution.items` traz `key` estável (`task:<id>` ou `checklist:<checklist-id>:<item-id>`), `type`, `parentKey`, `depth`, `name`, `done`, `state` (`done` ou `pending`) e `action.complete` com o comando do CLI e a chamada MCP que conclui o item. A ordem segue a tarefa, seus checklists e as subtarefas, pelo `orderindex`. `--raw` junto de `--markdown` sai com `2`.

`clickupfy task search`

Busca tarefas em todo o workspace associado ao perfil. É uma consulta paginada, limitada por `--max-pages`, que aceita de `1` a `100` páginas de 100 tarefas e usa `100` como padrão. O texto de `--query` procura ID, ID personalizado, nome ou descrição.

PARÂMETRO	OBRIGATÓRIO	USO
`--query <texto>`	não	Termo de busca em ID, nome ou descrição.

PARÂMETRO	OBRIGATÓRIO	USO
`--status <valores>`	não	Filtra por status, separados por vírgula.
`--assignee <valores>`	não	Filtra por responsáveis, separados por vírgula.
`--include-closed`	não	Inclui tarefas concluídas.
`--max-pages <n>`	não	Limite entre `1` e `100`; o padrão é `100`.

Exemplos:

```
clickupfy task search --query "autenticação"
```

```
clickupfy task search --query 86abc123
```

```
clickupfy task search --query "API" --status "in progress"
```

```
clickupfy task search --assignee 42 --include-closed --max-pages 5
```

```
clickupfy task search --query "checkout" --max-pages 10 --account cliente-a --json
```

Se mais de uma tarefa corresponder ao pedido, apresente os IDs e nomes para a pessoa escolher a correta antes de executar uma escrita. Em um MCP de projeto, a tool `clickupfy_tasks_search` percorre somente a List fixada; sem List fixada, percorre o workspace, como o CLI.

Criação e atualização de tarefas

`clickupfy task create`

Cria tarefa ou subtarefa na List indicada. `--name` é obrigatório e `--list` é obrigatória no CLI; no MCP a List fixada é o padrão. Use `--description` para texto simples ou `--markdown-content` quando a descrição precisa preservar Markdown. O comando não impõe uma regra de prioridade, status ou pontos do seu time; ele transmite valores válidos para a API.

PARÂMETRO	OBRIGATÓRIO	USO
`--list <id>`	condicional	List de destino.
`--name <nome>`	sim	Nome da tarefa.

PARÂMETRO	OBRIGATÓRIO	USO
`--description <texto>`	não	Descrição em texto simples.
`--markdown-content <markdown>`	não	Descrição em Markdown.
`--status <status>`	não	Status inicial existente na List.
`--priority <n>`	não	`1` urgente, `2` alta, `3` normal, `4` baixa.
`--assignee <valores>`	não	IDs numéricos das pessoas responsáveis, separados por vírgula.
`--parent <task-id>`	não	Cria como subtarefa da tarefa informada.
`--start-date <AAAA-MM-DD>`	não	Data de início, gravada no começo do dia no fuso local.
`--due-date <AAAA-MM-DD>`	não	Data de entrega, gravada no começo do dia no fuso local.
`--points <n>`	não	Sprint Points iguais ou maiores que zero.

Exemplos:

```
clickupfy task create --list 901328052926 --name "Corrigir retorno da API"
```

```
clickupfy task create \
  --list 901328052926 \
  --name "Documentar autenticação" \
  --description "Explicar o fluxo de login."
```

```
clickupfy task create \
  --list 901328052926 \
  --name "Criar testes" \
  --markdown-content "## Casos\n\n- Login válido\n- Senha inválida"
```

```
clickupfy task create \
  --list 901328052926 \
  --name "Implementar refresh token" \
  --status "in progress" \
  --priority 2 \
  --assignee 42,57 \
  --start-date 2026-11-16 \
  --due-date 2026-11-20 \
  --points 5
```

```
clickupfy task create \
  --list 901328052926 \
  --parent 86abc123 \
  --name "Cobrir expiração de sessão" \
  --markdown-content "Adicionar casos de teste para sessão expirada." \
  --account produto
```

As datas `--start-date` e `--due-date` viram o início do dia no fuso local da máquina e são enviadas com `start_date_time` e `due_date_time` iguais a `false`, para o ClickUp mostrar só a data. A versão 0.9.0 gravava meia-noite UTC, então uma tarefa criada agora pode aparecer com um horário diferente do que aparecia antes.

Consulte `clickupfy list get <list-id>` para copiar o status exato. Depois da criação, guarde o `id` devolvido e chame `task get` para confirmar pai, datas, responsáveis e conteúdo. `--parent` só define o pai na criação; ele não move uma tarefa existente.

`clickupfy task update <task-id>`

Atualiza campos de uma tarefa existente. Pelo menos um campo é obrigatório. `--clear-start-date` e `--clear-due-date` removem as respectivas datas; informar a flag de remoção e a data nova na mesma chamada sai com `2`.

PARÂMETRO	OBRIGATÓRIO	USO
`<task-id>`	sim	Tarefa a alterar.
`--name <nome>`	não	Novo nome.
`--description <texto>`	não	Nova descrição em texto simples.
`--markdown-content <markdown>`	não	Nova descrição em Markdown.
`--status <status>`	não	Status existente na List.
`--priority <n>`	não	Prioridade de `1` a `4`.

PARÂMETRO	OBRIGATÓRIO	USO
`--start-date <AAAA-MM-DD>`	não	Nova data de início, no começo do dia local.
`--clear-start-date`	não	Remove a data de início.
`--due-date <AAAA-MM-DD>`	não	Nova data de entrega, no começo do dia local.
`--clear-due-date`	não	Remove a data de entrega.
`--points <n>`	não	Sprint Points não negativos.

Exemplos:

```
clickupfy task update 86abc123 --status "in progress"
```

```
clickupfy task update 86abc123 --name "Corrigir autenticação por token"
```

```
clickupfy task update 86abc123 --priority 1 --points 8
```

```
clickupfy task update 86abc123 --start-date 2026-11-16 --due-date 2026-11-20
```

```
clickupfy task update 86abc123 \
  --markdown-content "## Entrega\n\nPublicar a documentação atualizada." \
  --clear-due-date \
  --account produto
```

`--description` e `--markdown-content` representam alternativas de conteúdo; envie apenas a forma que você quer persistir. Uma atualização de status precisa usar a grafia devolvida por `list get` . Releia a tarefa depois da escrita e compare cada campo solicitado com o estado devolvido.

`clickupfy task delete <task-id>`

Exclui permanentemente uma tarefa. Em terminal interativo, pede confirmação; em automação, exige `--yes` , senão sai com `2` . Leia nome, List e relação com Sprint antes de executar. A exclusão não é uma forma de retirar a tarefa de uma Sprint: use `sprint remove-task` para remover apenas a associação. Em JSON, `data` traz `{ deleted, taskId }` .

PARÂMETRO	OBRIGATÓRIO	USO
`<task-id>`	sim	Tarefa a excluir.

PARÂMETRO	OBRIGATÓRIO	USO
`--yes`	não	Confirma a exclusão sem prompt.

Exemplos:

```
clickupfy task delete 86abc123
```

```
clickupfy task delete 86abc123 --yes
```

```
clickupfy task delete 86abc123 --account produto --yes
```

```
clickupfy task delete 77def456 --yes --account cliente-a
```

```
clickupfy task get 86abc123 && clickupfy task delete 86abc123 --yes
```

O último exemplo lê a tarefa no mesmo terminal, mas ainda exige que você compare o resultado e tenha autorização explícita. Recusar a confirmação sai com `130`. No MCP, a tool equivalente exige o campo literal `confirm: true`.

Checklists e comentários

`clickupfy checklist create <task-id> --name <nome>`

Cria um checklist vazio em uma tarefa. Os itens são criados pelo comando seguinte. Use um nome que indique a finalidade, como `Testes`, `Publicação` ou `Revisão manual`.

PARÂMETRO	OBRIGATÓRIO	USO
`<task-id>`	sim	Tarefa proprietária.
`--name <nome>`	sim	Nome do checklist.

Exemplos:

```
clickupfy checklist create 86abc123 --name "Testes"
```

```
clickupfy checklist create 86abc123 --name "Revisão de segurança"
```

```
clickupfy checklist create 77def456 --name "Publicação"
```

```
clickupfy checklist create 86abc123 --name "Aceite" --account produto
```

```
clickupfy checklist create 86abc123 --name "Regressão" --json
```

Guarde o ID retornado em `{ checklist }`: `item-create` usa o ID do checklist, não o ID da tarefa. Crie o checklist na tarefa que realmente possui os itens; uma subtarefa pode ter checklist próprio e não deve receber os itens da tarefa pai por acidente.

`clickupfy checklist item-create <checklist-id> --name <nome>`

Acrescenta um item aberto a um checklist. `--assignee` é opcional e recebe um ID numérico. O comando não aceita o ID da tarefa porque o checklist já define seu proprietário.

PARÂMETRO	OBRIGATÓRIO	USO
`<checklist-id>`	sim	Checklist que receberá o item.
`--name <nome>`	sim	Texto do item.
`--assignee <id>`	não	ID numérico da pessoa responsável.

Exemplos:

```
clickupfy checklist item-create b955c4dc --name "Executar testes unitários"
```

```
clickupfy checklist item-create b955c4dc --name "Executar build"
```

```
clickupfy checklist item-create b955c4dc --name "Revisar documentação" --assignee 42
```

```
clickupfy checklist item-create 3f1a7c22 --name "Conferir changelog" --account
produto
```

```
clickupfy checklist item-create 3f1a7c22 --name "Validar publicação" --assignee 57
--json
```

Um item deve descrever uma verificação observável, não uma promessa genérica. Depois de criar itens, execute `task get --json` para localizar o `item-id` e confirmar que todos começam abertos. `--assignee` que não é inteiro sai com `2`.

`clickupfy checklist set <task-id> <checklist-id> <item-id>`

Marca um item como concluído com `--resolved` ou o reabre com `--open`. Escolha exatamente uma flag. O ClickUpfy primeiro confirma que o item pertence à tarefa raiz informada ou a uma subtarefa dela, grava o estado e relê a tarefa para confirmar o resultado.

PARÂMETRO	OBRIGATÓRIO	USO
`<task-id>`	sim	Tarefa raiz usada para validar a associação.
`<checklist-id>`	sim	Checklist proprietário.
`<item-id>`	sim	Item a alterar.
`--resolved`	condicional	Marca como concluído.
`--open`	condicional	Reabre o item.

Exemplos:

```
clickupfy checklist set 86abc123 b955c4dc 21e08dc8 --resolved
```

```
clickupfy checklist set 86abc123 b955c4dc 21e08dc8 --open
```

```
clickupfy checklist set 86abc123 3f1a7c22 8b2d7e10 --resolved --account produto
```

```
clickupfy checklist set 77def456 3f1a7c22 8b2d7e10 --resolved --json
```

```
clickupfy checklist set 77def456 3f1a7c22 8b2d7e10 --open
```

O `data` do `--json` traz o item na forma da fila de `task get`: `key`, `type`, `taskId`, `parentKey`, `depth`, `name`, `done`, `state`, `checklistId`, `checklistName` e `action`. Não use `--resolved --open` nem omita as duas: o comando sai com `2`. Item que não pertence ao checklist da tarefa sai com `6`, e estado não confirmado na releitura sai com `5`.

`clickupfy comment list --task <id>`

Lista os comentários de uma tarefa com autoria, data e texto, percorrendo todas as páginas da API. O ClickUpfy normaliza o conteúdo para o campo `text` da tabela compacta. Com `--json`, `data` traz `{ comments: [] }` com `id`, `user`, `date` e `text`.

PARÂMETRO	OBRIGATÓRIO	USO
`--task <id>`	sim	Tarefa cujos comentários serão lidos.

Exemplos:

```
clickupfy comment list --task 86abc123
```

```
clickupfy comment ls --task 86abc123
```

```
clickupfy comment list --task 86abc123 --json
```

```
clickupfy comment list --task 86abc123 --account produto
```

```
clickupfy comment list --task 77def456 --account cliente-a --json
```

Leia os comentários antes de atualizar status ou publicar outro progresso, pois podem conter uma orientação nova. Nunca publique credenciais, dados pessoais, URLs assinadas ou logs extensos em um comentário.

`clickupfy comment create --task <id> --text <texto>`

Publica um comentário em uma tarefa. `--notify-all` pede ao ClickUp que notifique todos os participantes. Use o comando para registrar início, resultado de teste, mudança de escopo ou conclusão, sempre com informação verificável.

PARÂMETRO	OBRIGATÓRIO	USO
`--task <id>`	sim	Tarefa que receberá o comentário.
`--text <texto>`	sim	Conteúdo do comentário.
`--notify-all`	não	Notifica os participantes da tarefa.

Exemplos:

```
clickupfy comment create --task 86abc123 --text "Iniciei a análise da tarefa."
```

```
clickupfy comment create --task 86abc123 --text "Os testes unitários passaram."
```

```
clickupfy comment create --task 86abc123 --text "Aguardando acesso ao ambiente de homologação." --notify-all
```

```
clickupfy comment create --task 86abc123 --text "Documentação e ebook foram
atualizados." --account produto
```

```
clickupfy comment create --task 77def456 --text "Validação final concluída." --
notify-all --json
```

O `data` do `--json` traz `{ id, hist_id, date }`. Reler `comment list` confirma que o comentário ficou na tarefa pretendida e que o texto chegou completo. O comando não modifica status, timer ou checklist. Obrigatória ausente sai com `2`.

Time tracking

`clickupfy time current`

Consulta o time entry em execução no workspace associado ao perfil. Chame este comando antes de `time start` para não iniciar outro registro sem saber qual tarefa já está em andamento. Sem cronômetro ativo, `data` traz `{ entry: null }` com exit code `0`.

Exemplos:

```
clickupfy time current
```

```
clickupfy time current --json
```

```
clickupfy time current --account produto
```

```
clickupfy time current --account cliente-a --json
```

```
clickupfy time current && clickupfy task get 86abc123
```

Se houver entrada ativa para outra tarefa, pare e peça orientação antes de usar `time stop`. O comando é somente leitura e não altera status ou comentários.

`clickupfy time start --task <id>`

Inicia um time entry associado à tarefa. A descrição é opcional e deve dizer o trabalho em curso. O registro pertence ao usuário autenticado pelo perfil e ao workspace desse perfil.

O ClickUp mantém um cronômetro por pessoa. Quando já existe um em outra tarefa, o ClickUp troca o cronômetro; antes de iniciar, o comando consulta o atual, avisa no stderr e devolve a tarefa anterior em `previous`, sem impedir a troca. `previous` traz `id`, `taskId`, `taskName`, `description` e `start`.

PARÂMETRO	OBRIGATÓRIO	USO
`--task <id>`	sim	Tarefa associada ao tempo.
`--description <texto>`	não	Descrição curta do trabalho.

Exemplos:

```
clickupfy time start --task 86abc123
```

```
clickupfy time start --task 86abc123 --description "Implementação"
```

```
clickupfy time start --task 86abc123 --description "Testes de regressão"
```

```
clickupfy time start --task 86abc123 --description "Revisão de documentação" --
account produto
```

```
clickupfy time start --task 77def456 --description "Correção da integração" --
account cliente-a
```

Em JSON, `data` traz `{ entry, previous? }`. Depois da chamada, execute `time current` para confirmar o item ativo. Iniciar um cronômetro não muda o status da tarefa nem avisa participantes; essas ações usam `task update` e `comment create` separadamente. `--task` ausente sai com `2`.

`clickupfy time stop`

Encerra o time entry em execução no workspace do perfil. Não recebe argumento de tarefa: ele para o registro atual. Por isso a consulta anterior é necessária em trabalhos paralelos ou quando mais de uma pessoa usa o mesmo perfil local.

Exemplos:

```
clickupfy time stop
```

```
clickupfy time stop --account produto
```

```
clickupfy time stop --account cliente-a
```

```
clickupfy time stop && clickupfy time current
```

```
clickupfy time stop --json
```

O quarto exemplo verifica que não há entrada em execução após o encerramento. Sem cronômetro ativo, o comando sai com `5` e a mensagem do ClickUp. Parar o tempo não fecha tarefa, não altera seus pontos e não resolve checklist items. Registre a conclusão do trabalho nas interfaces próprias se elas também forem autorizadas.

Tags de tarefas

`clickupfy task add-tag <task-id> <tag>`

Adiciona uma tag a uma tarefa existente pela API do ClickUp. As demais tags permanecem na tarefa. Cada execução adiciona um único nome; coloque nomes com espaços entre aspas. O comando não gerencia as cores ou o catálogo de tags do Space.

PARÂMETRO	TIPO	OBRIGATÓRIO	PADRÃO E USO
`<task-id>`	string	sim	ID da tarefa ou subtarefa.
`<tag>`	string	sim	Nome exato da tag, sem valor padrão; nomes vazios são recusados.
`--custom-task-id`	booleano	não	Desativado; quando presente, usa o workspace do perfil para resolver o ID personalizado.
`--account <perfil>`	string	não	Perfil ativo por padrão.

Exemplos:

Adicione uma tag pelo ID interno:

```
clickupfy task add-tag 86abc123 bug
```

Preserve um nome com espaço e acento:

```
clickupfy task add-tag 86abc123 "em revisão"
```

Use o workspace do perfil ativo com um ID personalizado:

```
clickupfy task add-tag DEV-42 backend --custom-task-id
```

Selecione o perfil para uma tarefa de outro workspace:

```
clickupfy task add-tag 77def456 documentação --account produto
```

Capture o resultado em um script e consulte a tarefa:

```
clickupfy --json task add-tag 86abc123 "QA / revisão"
```

O `data` do `--json` traz `{ added, taskId, tagName }`. Essa confirmação indica que a API aceitou a chamada; confira as tags persistidas com `task get --raw`. Para IDs personalizados, use o ID interno retornado pela tarefa nessa leitura. Erros de permissão, tarefa inexistente e outras recusas do ClickUp são propagados sem imprimir sucesso.

O endpoint usado é Add Tag To Task, com `POST /task/{task_id}/tag/{tag_name}`. Nomes e IDs são codificados na URL; IDs personalizados enviam `custom_task_ids=true` e `team_id` na query.

Gestão do catálogo de tags

As operações usam o perfil ativo ou `--account` no CLI e `account` no MCP. A API key precisa ter acesso ao Space ou à tarefa. Falhas do ClickUp são propagadas; no MCP, retornam `isError: true`. Ferramentas de escrita não ficam disponíveis em modo somente leitura. O MCP aceita `spaceId` ou usa o Space fixado na configuração; o CLI exige `--space` nos comandos `tag`.

`clickupfy tag list --space <id>`

Lista nomes e cores das tags de um Space. Com `--json`, `data` traz `{ tags: [] }` com `name`, `tag_fg` e `tag_bg`; o terminal mostra uma tabela. A consulta não altera tags.

`spaceId` é string, opcional quando o MCP fixa o Space. No CLI, `--space` é obrigatória.

Exemplos:

```
clickupfy tag list --space 90070363783
```

```
clickupfy tag list --space 90070363783 --json
```

```
clickupfy tag list --space 90070363783 --account dev
```

```
clickupfy tag list --space 9007154145 --account produto --json
```

```
clickupfy tag list --space 9007154145 --json > tags.json
```

Após uma escrita, consulte `tag list --space <id>` para conferir o catálogo ou `task get <task-id> --raw` para conferir as associações da tarefa.

`clickupfy tag create <name> --space <id>`

Cria uma tag no catálogo do Space com nome e cores. O `data` traz `{ created, spaceId, name }`. A associação a tarefas é feita separadamente com `task add-tag`.

`name`, `foreground` e `background` são obrigatórios. As cores usam `#RRGGBB`. No CLI, use `--foreground` e `--background`. Nome vazio e cor fora do formato são recusados com `2`.

Exemplos:

```
clickupfy tag create bug --space 90070363783 --foreground "#ffffff" --background
"#cc0000"
```

```
clickupfy tag create "em revisão" --space 90070363783 --foreground "#000000" --
background "#ffcc00"
```

```
clickupfy tag create backend --space 90070363783 --foreground "#ffffff" --
background "#112233" --json
```

```
clickupfy tag create documentação --space 9007154145 --foreground "#ffffff" --
background "#336699" --account produto
```

```
clickupfy tag create "QA / revisão" --space 90070363783 --foreground "#ffffff" --
background "#663399"
```

Sem `--space` no CLI, o comando sai com `2`. Após uma escrita, consulte `tag list --space <id>` para conferir o catálogo ou `task get <task-id> --raw` para conferir as associações da tarefa.

`clickupfy tag update <tag> --space <id>`

Renomeia a tag ou altera suas cores no Space. O comando consulta a definição atual e preserva os campos omitidos. O `data` traz `{ updated, spaceId, name }`. Tag ausente sai com `6`; atualização sem nenhum campo de alteração sai com `2`.

`tagName` é o nome atual obrigatório; no CLI, é o argumento `<tag>`. `name`, `foreground` e `background` são opcionais, com pelo menos um campo informado. No CLI, use `--name`, `--foreground` e `--background`.

Exemplos:

```
clickupfy tag update bug --space 90070363783 --name defeito
```

```
clickupfy tag update bug --space 90070363783 --foreground "#ffffff"
```

```
clickupfy tag update bug --space 90070363783 --background "#cc0000"
```

```
clickupfy tag update "em revisão" --space 90070363783 --name revisada --background
"#00aa00"
```

```
clickupfy tag update backend --space 9007154145 --foreground "#ffffff" --background
"#112233" --account produto --json
```

Após uma escrita, consulte `tag list --space <id>` para conferir o catálogo ou `task get <task-id> --raw` para conferir as associações da tarefa.

`clickupfy tag delete <tag> --space <id>`

Exclui a tag do Space e de todas as tarefas que a usavam. Consulte as tarefas que a utilizam antes de executar essa alteração. O `data` traz `{ deleted, spaceId, name }`. Para remover somente a associação a uma tarefa, use `task remove-tag`.

`tagName` é obrigatório. No MCP, `confirm: true` é obrigatório. No CLI, `--yes` dispensa o prompt; sem terminal interativo, a confirmação precisa ser explícita. Recusar a confirmação sai com `130`. O cliente consulta a definição da tag para enviar o corpo DELETE exigido pela API.

Exemplos:

```
clickupfy tag delete bug --space 90070363783 --yes
```

```
clickupfy tag delete "em revisão" --space 90070363783 --yes
```

```
clickupfy tag delete backend --space 90070363783 --yes --json
```

```
clickupfy tag delete documentação --space 9007154145 --yes --account produto
```

```
clickupfy tag delete "QA / revisão" --space 90070363783 --yes
```

Após uma escrita, consulte `tag list --space <id>` para conferir o catálogo ou `task get <task-id> --raw` para conferir as associações da tarefa.

`clickupfy task remove-tag <task-id> <tag>`

Remove uma tag da tarefa e mantém a definição no Space. O `data` traz `{ removed, taskId, tagName }`. As outras associações da tarefa permanecem intactas.

`taskId` e `tagName` são obrigatórios. `customTaskId` é booleano opcional, desativado por padrão; no CLI, use `--custom-task-id`. IDs personalizados usam o workspace do perfil.

Exemplos:

```
clickupfy task remove-tag 86abc123 bug
```

```
clickupfy task remove-tag 86abc123 "em revisão"
```

```
clickupfy task remove-tag DEV-42 backend --custom-task-id
```

```
clickupfy task remove-tag 77def456 documentação --account produto --json
```

```
clickupfy task remove-tag sub-1 "QA / revisão"
```

Após uma escrita, consulte `tag list --space <id>` para conferir o catálogo ou `task get <task-id> --raw` para conferir as associações da tarefa.

Planeje Sprints e Sprint Points

Como o ClickUpfy reconhece uma Sprint

No ClickUp, uma Sprint é uma List dentro de um Sprint Folder. O ClickUpfy considera Sprint uma List que possua `start_date` e `due_date`. Lists comuns do mesmo Folder ficam fora do resultado, salvo quando `--include-regular` for usado.

O status (`planned`, `active` ou `completed`) é calculado contra a data de referência: hoje ou o valor de `--at <AAAA-MM-DD>`.

O CLI não cria Sprints nem habilita o Sprint ClickApp. Essas operações continuam na interface do ClickUp.

Liste os ciclos

```
clickupfy sprint list --folder <sprint-folder-id>
```

Para diagnóstico de um Folder misto:

```
clickupfy sprint list \
  --folder <sprint-folder-id> \
  --include-regular
```

Confira nome, ID, início e término. Um período ausente indica uma List comum ou uma Sprint ainda não configurada corretamente. `--archived` inclui Lists arquivadas.

Encontre a Sprint atual

```
clickupfy sprint current --folder <sprint-folder-id>
```

O comando espera uma única Sprint ativa na data atual. Nenhuma Sprint ativa ou mais de uma sobreposta produz uma falha explícita. Corrija o calendário no ClickUp antes de automatizar a seleção.

Para calcular o estado em outra data, use `--at`:

```
clickupfy sprint current --folder <sprint-folder-id> --at 2026-11-13
```

Leia o relatório

```
clickupfy sprint get <sprint-id>
clickupfy sprint tasks <sprint-id>
clickupfy sprint tasks <sprint-id> --open-only
```

O relatório percorre todas as páginas e inclui tarefas concluídas. Ele traz `totalTasks`, `completedTasks`, `openTasks`, `totalPoints`, `completedPoints`, `unestimatedTasks`, `taskCompletionPercent`, `pointsCompletionPercent` e `tasksByStatus`.

O avanço é calculado por quantidade de tarefas e por Sprint Points. Uma tarefa sem Points participa da quantidade de itens, mas não acrescenta peso ao cálculo por pontos.

Use `--open-only` para a fila de trabalho corrente, não para o relatório final da Sprint.

Associe uma tarefa sem mover a List principal

```
clickupfy sprint add-task <sprint-id> <task-id>
```

A API associa a tarefa à Sprint e preserva sua List principal. Isso permite manter backlog e ciclo como dimensões diferentes.

Para remover a associação:

```
clickupfy sprint remove-task <sprint-id> <task-id>
```

Essa ação não exclui a tarefa.

Defina Sprint Points

```
clickupfy sprint set-points <task-id> 5
```

O mesmo campo pode ser atualizado pelo grupo de tarefas:

```
clickupfy task update <task-id> --points 8
```

Use a escala adotada pela equipe. O ClickUpfy transmite o número, mas não define se ele representa complexidade, esforço ou tamanho relativo.

Configure o MCP para Sprints

Acrescente o Folder ao projeto:

```
clickupfy --account produto agent install \
  --space 10 \
  --folder 20 \
  --list 30 \
  --sprint-folder 40
```

As ferramentas de Sprint podem omitir `folderId` quando esse valor está fixado. Um ID diferente é recusado. Consultas permanecem disponíveis em read-only. Associação e Points aparecem apenas no servidor com escrita.

Depois do planejamento, aprenda a registrar o tempo.

Referência do CLI: Sprints e Sprint Points

Consulta de ciclos

`clickupfy sprint list --folder <id>`

Lista Sprints identificadas pelo período. `--folder` é obrigatória no CLI; no MCP, o Sprint Folder fixado é o padrão. Use `--include-regular` para diagnosticar um Folder misto e `--archived` para incluir Lists arquivadas.

PARÂMETRO	OBRIGATÓRIO	USO
`--folder <id>`	sim	Sprint Folder consultado.
`--archived`	não	Inclui Lists arquivadas.
`--include-regular`	não	Inclui Lists sem período configurado.
`--at <AAAA-MM-DD>`	não	Data usada no cálculo do estado.

Exemplos:

```
clickupfy sprint list --folder 4001
```

```
clickupfy sprint list --folder 4001 --at 2026-11-13
```

```
clickupfy sprint list --folder 4001 --include-regular
```

```
clickupfy sprint list --folder 4001 --archived --include-regular
```

```
clickupfy sprint list --folder 4001 --at 2026-12-01 --account produto --json
```

Em `--json`, `data` traz `{ sprints: [] }` e cada Sprint tem `id`, `name`, `startDate`, `dueDate`, `status`, `taskCount` e `isSprint`. Uma List sem início ou término aparece somente com `--include-regular`; ela não deve ser tratada como Sprint. Guarde o ID de uma Sprint retornada para os comandos de relatório e associação. Sem `--folder`, o comando sai com `2`.

`clickupfy sprint current --folder <id>`

Encontra a única Sprint cujo período inclui a data atual ou a data indicada por `--at`. Sem Sprint ativa, o comando sai com `6`. Com mais de uma Sprint ativa, sai com `2`, lista as candidatas na mensagem e devolve `error.details.candidates[]` no JSON. Essa falha protege o planejamento contra uma seleção arbitrária.

PARÂMETRO	OBRIGATÓRIO	USO
`--folder <id>`	sim	Sprint Folder consultado.
`--at <AAAA-MM-DD>`	não	Data usada para localizar a Sprint.

Exemplos:

```
clickupfy sprint current --folder 4001
```

```
clickupfy sprint current --folder 4001 --at 2026-11-13
```

```
clickupfy sprint current --folder 4001 --at 2026-12-01
```

```
clickupfy sprint current --folder 4001 --account produto
```

```
clickupfy sprint current --folder 5001 --at 2026-10-15 --account cliente-a --json
```

Corrija datas sobrepostas no ClickUp quando o comando falhar. Não passe uma List comum para `sprint get` como alternativa: o relatório depende do período da Sprint.

`clickupfy sprint get <sprint-id>`

Mostra período, estado temporal, total de tarefas, distribuição por status e progresso por quantidade e por Sprint Points. `--at` muda a leitura temporal, sem alterar a Sprint nem suas tarefas.

PARÂMETRO	OBRIGATÓRIO	USO
`<sprint-id>`	sim	Sprint a analisar.
`--at <AAAA-MM-DD>`	não	Data usada no cálculo do estado.

Exemplos:

```
clickupfy sprint get 901328052926
```

```
clickupfy sprint get 901328052926 --at 2026-11-13
```

```
clickupfy sprint get 901328060001
```

```
clickupfy sprint get 901328052926 --account produto
```

```
clickupfy sprint get 901328060002 --at 2026-12-01 --account cliente-a --json
```

O `data` traz a Sprint com `totalTasks`, `completedTasks`, `openTasks`, `totalPoints`, `completedPoints`, `unestimatedTasks`, `taskCompletionPercent`, `pointsCompletionPercent` e `tasksByStatus`. Uma tarefa sem Points entra no total de tarefas, mas não adiciona peso ao progresso por pontos. Consulte `sprint tasks` para inspecionar as tarefas que compõem esses números. Uma List sem período no lugar de `<sprint-id>` sai com `2`.

`clickupfy sprint tasks <sprint-id>`

Lista tarefas associadas à Sprint. Por padrão, inclui tarefas concluídas. `--open-only` omite itens fechados e serve para a fila de trabalho atual, não para conferir o resultado final do ciclo.

PARÂMETRO	OBRIGATÓRIO	USO
`<sprint-id>`	sim	Sprint consultada.
`--open-only`	não	Omite tarefas concluídas.

Exemplos:

```
clickupfy sprint tasks 901328052926
```

```
clickupfy sprint tasks 901328052926 --open-only
```

```
clickupfy sprint tasks 901328060001
```

```
clickupfy sprint tasks 901328052926 --open-only --account produto
```

```
clickupfy sprint tasks 901328060002 --account cliente-a --json
```

O CLI devolve `{ tasks: [] }` com as tarefas completas da API. As tools MCP devolvem linhas compactas com `id`, `custom_id`, `name`, `status`, `priority`, `points`, `assignees`, `start_date`, `due_date` e `url`. Para obter descrição, subtarefas e checklist items de uma tarefa, execute `clickupfy task get <task-id>`.

Alterações no planejamento

`clickupfy sprint add-task <sprint-id> <task-id>`

Associa uma tarefa existente a uma Sprint sem trocar sua List principal. Isso permite manter a tarefa no backlog e no ciclo atual ao mesmo tempo. A operação não cria cópia, não move a tarefa e não modifica status. O `data` traz `{ sprintId, taskId, added }`.

PARÂMETRO	OBRIGATÓRIO	USO
`<sprint-id>`	sim	Sprint de destino.
`<task-id>`	sim	Tarefa que será associada.

Exemplos:

```
clickupfy sprint add-task 901328052926 86abc123
```

```
clickupfy sprint add-task 901328052926 77def456
```

```
clickupfy sprint add-task 901328052926 86abc123 --account produto
```

```
clickupfy sprint add-task 901328060002 55ghi789 --account cliente-a
```

```
clickupfy sprint add-task 901328052926 99jkl012 --json
```

Reler `sprint tasks <sprint-id>` confirma a associação. Só execute essa ação quando o planejamento da Sprint estiver no escopo autorizado.

`clickupfy sprint remove-task <sprint-id> <task-id>`

Remove a associação entre tarefa e Sprint. A tarefa continua existindo na List principal e não perde seus comentários, checklist, tempo ou histórico. O `data` traz `{ sprintId, taskId, removed }`.

PARÂMETRO	OBRIGATÓRIO	USO
`<sprint-id>`	sim	Sprint cuja associação será removida.
`<task-id>`	sim	Tarefa associada.

Exemplos:

```
clickupfy sprint remove-task 901328052926 86abc123
```

```
clickupfy sprint remove-task 901328052926 77def456
```

```
clickupfy sprint remove-task 901328052926 86abc123 --account produto
```

```
clickupfy sprint remove-task 901328060002 55ghi789 --account cliente-a
```

```
clickupfy sprint remove-task 901328052926 99jkl012 --json
```

Para excluir uma tarefa, use `task delete` apenas com autorização explícita. Para confirmar que a associação saiu, releia a Sprint e a tarefa depois desta operação.

`clickupfy sprint set-points <task-id> <points>`

Define Sprint Points de uma tarefa. O número deve ser igual ou maior que zero. O ClickUpfy não interpreta a escala: a equipe define se `1`, `2`, `3`, `5` ou outros valores representam tamanho, esforço ou complexidade.

PARÂMETRO	OBRIGATÓRIO	USO
`<task-id>`	sim	Tarefa que receberá pontos.
`<points>`	sim	Número não negativo.

Exemplos:

```
clickupfy sprint set-points 86abc123 0
```

```
clickupfy sprint set-points 86abc123 1
```

```
clickupfy sprint set-points 86abc123 3
```

```
clickupfy sprint set-points 86abc123 5 --account produto
```

```
clickupfy sprint set-points 77def456 8 --account cliente-a --json
```

Pontos negativos saem com `2`, sem chamar a API. O `data` traz a tarefa atualizada. `task update <task-id> --points <n>` altera o mesmo campo; use apenas uma das interfaces por alteração e releia a tarefa para confirmar o valor salvo.

Registre tempo de trabalho

Consulte antes de iniciar

```
clickupfy time current
```

Essa leitura evita iniciar um segundo registro enquanto outro item permanece em execução. Se houver um time entry ativo, confirme a tarefa e decida se ele deve continuar ou ser encerrado. Sem cronômetro, a resposta traz `entry: null` e sai com exit code 0.

Inicie um registro

```
clickupfy time start \
  --task <task-id> \
  --description "Implementação e testes"
```

Use uma descrição curta e observável. O registro pertence ao usuário autenticado pelo perfil selecionado.

O ClickUp mantém um cronômetro por pessoa. Se outro cronômetro estiver ativo em outra tarefa, `time start` troca o cronômetro, avisa no stderr e devolve a tarefa anterior em `previous`, com `id`, `taskId`, `taskName`, `description` e `start`. A troca não é impedida.

Em um fluxo com agente, inicie o tempo somente quando a implementação realmente começar. Leitura inicial, esclarecimento e espera por autorização não devem ser registrados automaticamente sem uma regra explícita da equipe.

Encerre o registro

```
clickupfy time stop
```

Depois, consulte novamente:

```
clickupfy time current
```

A ausência de registro em execução confirma o encerramento. `time stop` sem cronômetro ativo sai com exit code 5 e a mensagem do ClickUp.

Use outro perfil em uma operação

```
clickupfy --account cliente-a time current
clickupfy --account cliente-a time start \
  --task <task-id> \
  --description "Correção e regressão"
```

Essa seleção não altera o perfil ativo. É útil quando dois projetos usam workspaces diferentes na mesma máquina.

Relação com comentários e status

Time tracking não muda status nem publica comentário. As três ações representam evidências diferentes:

- status indica a etapa no fluxo
- comentário registra um evento legível pela equipe
- time entry mede o período atribuído ao trabalho.

A skill de implementação coordena essas ações, mas cada mutação continua explícita e verificável.

Continue em skills para agentes.

Use as skills para agentes

O catálogo incluído

O binário distribui cinco skills:

SKILL	RESPONSABILIDADE
`clickupfy-setup`	Configuração de perfil, skills e MCP do projeto.
`clickupfy-dev`	Consulta e atualização do trabalho diário de software.
`clickup-issue-create`	Criação de tarefa, subtarefas e checklists sem executar a implementação.
`clickup-issue-implement`	Execução acompanhada por plano, comentários, tempo, testes e status.
`clickupfy-release`	Preparação e acompanhamento de versões e artefatos.

As skills são instruções. Elas usam o CLI ou o MCP disponível, mas não contêm credenciais.

Inspecione antes de instalar

```
clickupfy agent skill list
clickupfy agent skill show clickupfy-dev
```

`list` apresenta o catálogo com nome e descrição. `show` imprime a fonte embutida da skill, útil para revisar permissões e fluxo antes da instalação. As cinco skills são mantidas em português do Brasil.

Configure o projeto

Use `$clickupfy-setup` para preparar um repositório novo ou revisar uma integração existente. A skill executa `clickupfy doctor`, orienta o perfil com `clickupfy setup` quando necessário, instala as skills por `npx skills add` e cria o MCP somente depois de confirmar os IDs do ClickUp.

Instale no projeto

Na raiz do repositório, o ClickUpfy delega a instalação ao gerenciador `skills`:

```
clickupfy agent skill install
```

As skills entram em `.agents/skills/` e o gerenciador registra a origem em `skills-lock.json`. Esse modo mantém a configuração próxima ao projeto e permite versionar as instruções.

Para instalar no catálogo global do usuário, em `~/.codex/skills/`:

```
clickupfy agent skill install --global
```

Para instalar só algumas skills, informe os nomes:

```
clickupfy agent skill install clickupfy-dev clickup-issue-implement
```

Prefira a instalação local quando projetos usam versões ou regras diferentes. Consulte a instalação efetiva com:

```
npx skills list
npx skills list --global
```

`--force` é aceito por compatibilidade. A atualização passa pelo próprio `npx skills add`, que mantém o lockfile e os caminhos canônicos.

Para instalar as skills e, no mesmo passo, gravar o servidor MCP do projeto:

```
clickupfy agent install --space 10 --list 30
```

`agent init` é o alias do mesmo comando.

Crie uma issue sem implementá-la

Peça explicitamente a skill de criação:

```
Use $clickup-issue-create para transformar esta solicitação em uma tarefa no
ClickUp. Crie subtarefas e checklists de teste, mas não implemente o trabalho.
```

A skill deve obter os status da List, redigir a descrição em Markdown, criar a tarefa principal e materializar subtarefas e checklists recursivos. A criação não autoriza editar código nem marcar validações como concluídas.

Implemente uma issue existente

```
Use $clickup-issue-implement para executar a tarefa <task-id> até a validação
final.
```

O fluxo lê a tarefa inteira, monta um plano visível, registra início, datas e time tracking quando aplicável, percorre subtarefas e checklist items e publica checkpoints. Um item de teste só é resolvido depois do comando correspondente passar.

A conclusão exige releitura do estado remoto, regressão aplicável e um status terminal existente na List.

Use a skill diária

`clickupfy-dev` é adequada para consultas e mutações pontuais:

```
Use $clickupfy-dev para ler a tarefa <task-id>, publicar o resultado dos testes
e marcar apenas o checklist correspondente.
```

Escopo e autorização continuam limitados pelo pedido. A skill não transforma uma consulta em implementação completa.

Prepare uma release

`clickupfy-release` lê a documentação de release do próprio repositório e acompanha versão, changelog, validações e artefatos. Ela não deve ser instalada como regra genérica em projetos que apenas consomem o CLI.

Leia a documentação embutida

Um agente sem acesso ao repositório pode ler a documentação pelo próprio binário:

```
clickupfy --skill
clickupfy --skill mcp
clickupfy skill cli
```

Agora veja como essas skills acessam o ClickUp pelo servidor MCP.

Referência do CLI: Docs, skills e servidor MCP

Docs

`clickupfy doc list`

Busca Docs no workspace associado ao perfil, seguindo o cursor da API. `--max-pages` controla quantas páginas de cursor são lidas e aceita inteiros de `1` a `50`; o padrão é `50`.

PARÂMETRO	OBRIGATÓRIO	USO
`--query <texto>`	não	Filtro pelo nome ou ID do Doc.
`--parent-id <id>`	não	Local pai do Doc.
`--parent-type <n>`	não	`4` Space, `5` Folder, `6` List, `7` Everything, `12` tarefa.
`--deleted`	não	Inclui Docs excluídos.
`--archived`	não	Inclui Docs arquivados.
`--creator <id>`	não	ID numérico da pessoa criadora.
`--max-pages <n>`	não	Cursor de `1` a `50`; padrão `50`.

Exemplos:

```
clickupfy doc list
```

```
clickupfy doc list --query "API"
```

```
clickupfy doc list --parent-id 901328052926 --parent-type 6
```

```
clickupfy doc list --archived --deleted --creator 42 --max-pages 10
```

```
clickupfy doc list --query "Manual" --max-pages 5 --account produto --json
```

`--parent-type` sem `--parent-id` não cria uma consulta útil, pois não há local para classificar. Com `--json`, `data` traz `{ docs: [] }`, com os metadados necessários para chamar `doc get` ou navegar pelas páginas.

`clickupfy doc get <doc-id>`

Obtém os metadados de um Doc no workspace atual. O comando não devolve o conteúdo das páginas; use `doc page list` para o conteúdo completo ou `doc page get` para uma página determinada.

Exemplos:

```
clickupfy doc get 8cdu8-123
```

```
clickupfy doc get 8cdu8-123 --json
```

```
clickupfy doc get 8cdu8-123 --account produto
```

```
clickupfy doc get 8cdu8-456 --account cliente-a --json
```

```
clickupfy doc get 8cdu8-123 --json > /tmp/doc-8cdu8-123.json
```

O último exemplo guarda o objeto completo. Para processamento por outro programa, use `--json` e preserve os dados do workspace como informação interna da equipe. Doc inexistente sai com `6`.

`clickupfy doc create --name <nome>`

Cria um Doc no workspace. Quando `--parent-id` é informado, `--parent-type` é obrigatório. Os tipos são `4` Space, `5` Folder, `6` List, `7` Everything e `12` tarefa. `--visibility` aceita `PRIVATE` ou `PUBLIC`; `--create-page` cria a primeira página em branco.

PARÂMETRO	OBRIGATÓRIO	USO
`--name <nome>`	sim	Nome do Doc.
`--parent-id <id>`	não	Local onde o Doc será criado.
`--parent-type <n>`	condicional	Tipo do local quando há `--parent-id`.
`--visibility <valor>`	não	Visibilidade, `PRIVATE` ou `PUBLIC`.
`--create-page`	não	Cria uma primeira página vazia.

Exemplos:

```
clickupfy doc create --name "Manual da API"
```

```
clickupfy doc create --name "Guia do produto" --create-page
```

```
clickupfy doc create --name "Notas da List" --parent-id 901328052926 --parent-type 6
```

```
clickupfy doc create --name "Documento interno" --parent-id 2001 --parent-type 5 --
visibility PRIVATE
```

```
clickupfy doc create --name "Plano de release" --parent-id 86abc123 --parent-type
12 --create-page --account produto --json
```

Uma criação bem-sucedida devolve o Doc com o novo ID. Releia seus metadados e crie as páginas pelo grupo `doc page`. A API pública usada pelo ClickUpfy não expõe exclusão de Docs, mudança de permissões ou reordenação da árvore.

`clickupfy doc page tree <doc-id>`

Mostra somente a árvore de páginas, sem o corpo, indentada no terminal. É a leitura mais econômica para descobrir um `page-id` e a relação pai-filho antes de criar uma subpágina ou atualizar conteúdo.

Exemplos:

```
clickupfy doc page tree 8cdu8-123
```

```
clickupfy doc page tree 8cdu8-123 --json
```

```
clickupfy doc page tree 8cdu8-123 --account produto
```

```
clickupfy doc page tree 8cdu8-456 --account cliente-a --json
```

```
clickupfy doc page tree 8cdu8-123 > /tmp/doc-8cdu8-123-tree.txt
```

Com `--json`, `data` traz `{ pages: [] }` com as páginas aninhadas. O retorno não é cópia de segurança do conteúdo. Use `doc page list` quando o objetivo for ler todas as páginas em Markdown.

`clickupfy doc page list <doc-id>`

Lista as páginas com conteúdo. `--max-page-depth` limita a profundidade de subpáginas retornada. `--content-format` aceita `text/md`, o padrão, ou `text/plain` para texto sem a representação Markdown da API.

PARÂMETRO	OBRIGATÓRIO	USO
`<doc-id>`	sim	Doc cujas páginas serão lidas.
`--max-page-depth <n>`	não	Profundidade máxima de subpáginas.
`--content-format <valor>`	não	`text/md` ou `text/plain`.

Exemplos:

```
clickupfy doc page list 8cdu8-123
```

```
clickupfy doc page ls 8cdu8-123 --max-page-depth 1
```

```
clickupfy doc page list 8cdu8-123 --max-page-depth 3
```

```
clickupfy doc page list 8cdu8-123 --content-format text/plain
```

```
clickupfy doc page list 8cdu8-123 --content-format text/md --account produto --json
```

Use uma profundidade pequena quando só precisa dos capítulos principais. Para uma página conhecida, `doc page get` reduz a resposta e evita transmitir o conteúdo de outras páginas a um agente.

`clickupfy doc page get <doc-id> <page-id>`

Obtém uma página específica e seu conteúdo. Em modo texto, o comando imprime somente o conteúdo. `--content-format` tem os mesmos valores de `doc page list`: `text/md` por padrão e `text/plain` como opção.

Exemplos:

```
clickupfy doc page get 8cdu8-123 8cdu8-456
```

```
clickupfy doc page get 8cdu8-123 8cdu8-456 --content-format text/plain
```

```
clickupfy doc page get 8cdu8-123 8cdu8-456 --json
```

```
clickupfy doc page get 8cdu8-123 8cdu8-789 --content-format text/md --account
produto
```

```
clickupfy doc page get 8cdu8-456 8cdu8-999 --account cliente-a --json
```

O ID da página deve pertencer ao Doc informado. Localize-o com `doc page tree` ou `doc page list`, não com tentativa e erro. Página inexistente sai com `6`.

`clickupfy doc page create <doc-id> --name <nome>`

Cria página ou subpágina em um Doc. `--parent-page` indica uma página pai; `--orderindex` escolhe a posição entre páginas irmãs. O conteúdo inicial e o subtítulo são opcionais. Não use `--orderindex` como substituto de uma revisão da árvore existente: leia a árvore antes de inserir conteúdo.

PARÂMETRO	OBRIGATÓRIO	USO
`<doc-id>`	sim	Doc proprietário.
`--name <nome>`	sim	Título da página.
`--content <texto>`	não	Conteúdo inicial.
`--file <caminho>`	não	Lê o conteúdo de um arquivo; use `-` para o stdin. Substitui `--content`.
`--sub-title <texto>`	não	Subtítulo.
`--parent-page <id>`	não	Página pai de uma subpágina.
`--orderindex <n>`	não	Posição entre páginas irmãs.
`--content-format <valor>`	não	`text/md` ou `text/plain`.

Exemplos:

```
clickupfy doc page create 8cdu8-123 --name "Introdução"
```

```
clickupfy doc page create 8cdu8-123 --name "Autenticação" --content
"Use uma API key pessoal."
```

```
clickupfy doc page create 8cdu8-123 --name "Plano" --file plan.md
```

```
clickupfy doc page create 8cdu8-123 --name "Erros" --parent-page 8cdu8-456 --
orderindex 2
```

```
clickupfy doc page create 8cdu8-123 --name "Integração" --content "# MCP" --content-
format text/md --account produto --json
```

Leia a página criada com `doc page get` e a árvore com `doc page tree` para confirmar conteúdo e posição. A criação não atualiza outras páginas nem cria um Doc novo.

`clickupfy doc page update <doc-id> <page-id>`

Atualiza título, subtítulo e/ou conteúdo de uma página. Pelo menos um campo é necessário, senão o comando sai com `2`. `--content-edit-mode` controla o conteúdo: `replace` substitui, `append` acrescenta ao fim e `prepend` insere no começo. Sem a opção, o modo é `replace`.

PARÂMETRO	OBRIGATÓRIO	USO
`<doc-id>`, `<page-id>`	sim	Doc e página a atualizar.
`--name <nome>`	não	Novo título.
`--sub-title <texto>`	não	Novo subtítulo.
`--content <texto>`	não	Novo conteúdo.
`--file <caminho>`	não	Lê o conteúdo de um arquivo; use `-` para o stdin. Substitui `--content`.
`--content-edit-mode <modo>`	não	`replace`, `append` ou `prepend`.
`--content-format <valor>`	não	`text/md` ou `text/plain`.

Exemplos:

```
clickupfy doc page update 8cdu8-123 8cdu8-456 --name "Visão geral"
```

```
clickupfy doc page update 8cdu8-123 8cdu8-456 --sub-title "Instalação e
configuração"
```

```
clickupfy doc page update 8cdu8-123 8cdu8-456 --content "# Novo conteúdo"
```

```
clickupfy doc page update 8cdu8-123 8cdu8-456 --content "\n## Changelog" --content-edit-mode append
```

```
clickupfy doc page update 8cdu8-123 8cdu8-789 --content "# Aviso\n\n" --content-edit-mode prepend --content-format text/md --account produto --json
```

Leia a página depois da escrita para confirmar o modo aplicado. Também é possível enviar o conteúdo pelo stdin:

```
cat log.md | clickupfy doc page update 8cdu8-123 8cdu8-456 --file - --content-edit-mode append
```

A API pública usada pelo ClickUpfy não oferece exclusão de página nem movimentação na árvore.

Skills e projeto de agente

`clickupfy agent skill list`

Lista as skills embutidas no binário: `clickupfy-setup`, `clickupfy-dev`, `clickup-issue-create`, `clickup-issue-implement` e `clickupfy-release`. O comando não instala nada e não precisa de perfil configurado. Com `--json`, `data` traz `{ skills: [] }` com `name` e `description`.

Exemplos:

```
clickupfy agent skill list
```

```
clickupfy agent skill list | sort
```

```
clickupfy agent skill list > /tmp/clickupfy-skills.txt
```

```
clickupfy agent skill list | wc -l
```

```
clickupfy agent skill list && clickupfy agent skill show clickupfy-dev
```

O catálogo é incorporado ao binário em tempo de build. `list` não verifica se uma skill já está instalada no projeto; ele mostra o que a versão instalada do ClickUpfy pode distribuir. As instruções e os metadados distribuídos estão em português do Brasil.

`clickupfy agent skill show <skill>`

Imprime o `SKILL.md` de uma skill embutida. Use o comando para revisar instruções e permissões antes da instalação. O argumento precisa ser um dos cinco nomes do catálogo. Com `--json`, `data` traz `{ name, content }`.

Exemplos:

```
clickupfy agent skill show clickupfy-dev
```

```
clickupfy agent skill show clickupfy-setup
```

```
clickupfy agent skill show clickup-issue-create
```

```
clickupfy agent skill show clickupfy-release
```

```
clickupfy agent skill show clickupfy-dev > /tmp/clickupfy-dev-skill.md
```

Uma skill desconhecida é recusada com `2`. O comando só lê o asset distribuído; ele não abre uma cópia local já instalada, que pode ter sido alterada pelo projeto.

`clickupfy agent skill install [skills...]`

Instala todas as skills quando nenhum nome é informado, ou somente as skills posicionais escolhidas, chamando `npx skills add promovaweb/clickupfy`. Sem `--global`, o destino é o catálogo de skills do projeto atual, e o gerenciador atualiza o `skills-lock.json`. `--global` instala no catálogo do usuário.

O comando roda `npx --yes skills add promovaweb/clickupfy --yes --copy --agent codex --skill <nome>...` e confere o resultado com `npx skills list --json`. `--force` existe apenas por compatibilidade com o 0.9.0, porque reinstalar é sempre permitido.

PARÂMETRO	OBRIGATÓRIO	USO
`[skills...]`	não	Uma ou mais skills do catálogo.
`--global`	não	Instala no catálogo global do usuário.
`--force`	não	Mantido por compatibilidade.

Exemplos:

```
clickupfy agent skill install
```

```
clickupfy agent skill install clickupfy-dev
```

```
clickupfy agent skill install clickup-issue-create clickup-issue-implement
```

```
clickupfy agent skill install --global clickupfy-dev
```

```
clickupfy agent skill install clickupfy-release --force
```

Skill desconhecida sai com `6`. Falha do `npx skills` ou `npx` ausente do `PATH` sai com `1`. Os caminhos canônicos pertencem ao gerenciador `npx skills`, por isso o ClickUpfy não aceita um `--target` próprio. Revise a fonte com `agent skill show` e consulte `npx skills list` antes de atualizar regras de um projeto.

`clickupfy agent install`

Instala todas as skills e grava um servidor `promovaweb-clickupfy` no `.mcp.json` e no `.codex/config.toml` do projeto atual. `--space` e `--list` são obrigatórias. `clickupfy agent init` é um alias compatível.

A entrada usa `mcp serve --account <perfil> --space <id> --list <id>` e acrescenta os IDs opcionais informados. A API key nunca vai para os arquivos: só o identificador do perfil é gravado. Uma entrada `promovaweb-clickupfy` já existente é substituída e as demais configurações são preservadas.

PARÂMETRO	OBRIGATÓRIO	USO
`--global`	não	Instala as skills globalmente.
`--workspace <id>`	não	Workspace esperado pelo MCP; precisa ser o do perfil.
`--space <id>`	sim	Space fixado para o projeto.
`--folder <id>`	não	Folder fixado.
`--list <id>`	sim	List fixa do projeto.
`--sprint-folder <id>`	não	Sprint Folder do projeto.
`--force`	não	Mantido por compatibilidade; as skills existentes são atualizadas.

Exemplos:

```
clickupfy agent install --space 90070363783 --list 901328052926
```

```
clickupfy agent install --space 90070363783 --folder 2001 --list 901328052926 --
account produto
```

```
clickupfy agent install --space 90070363783 --list 901328052926 --sprint-folder 4001
```

```
clickupfy agent install --global --space 90070363783 --list 901328052926 --force
```

```
clickupfy agent init --account cliente-a --workspace 9007154145 --space
901328060001 --list 901328060002 --sprint-folder 4002 --force
```

O `data` do `--json` traz `{ account, name, args, skills, files }`. Falta de `--space` ou `--list`, e um `--workspace` diferente do perfil, saem com `2`. Perfil inexistente sai com `6`. Reabra o cliente MCP e chame `clickupfy_mcp_context` para conferir o destino.

`clickupfy mcp serve --list <id>`

Inicia o servidor MCP em stdio. A saída padrão é reservada ao protocolo JSON-RPC; mensagens operacionais vão para o stderr. `--list` fixa a List do projeto e os outros IDs fixam os níveis superiores. `--read-only` remove as tools de escrita de `tools/list` e `--describe` lista as tools do escopo e encerra.

PARÂMETRO	OBRIGATÓRIO	USO
`--account <perfil>`	não	Perfil fixado.
`--workspace <id>`	não	Workspace esperado.
`--space <id>`	não	Space fixado.
`--folder <id>`	não	Folder fixado.
`--list <id>`	não	List fixa do projeto.
`--sprint-folder <id>`	não	Sprint Folder fixado.
`--read-only`	não	Expõe somente ferramentas de leitura.
`--describe`	não	Lista as tools do escopo e sai.

Exemplos:

```
clickupfy mcp serve --list 901328052926
```

```
clickupfy mcp serve --account produto --list 901328052926
```

```
clickupfy mcp serve --workspace 90070363783 --space 90070363783 --folder 2001 --
list 901328052926
```

```
clickupfy mcp serve --list 901328052926 --sprint-folder 4001 --read-only
```

```
clickupfy mcp serve --account cliente-a --space 901328060001 --list 901328060002 --
describe
```

Normalmente o cliente MCP executa esse comando a partir do `.mcp.json` ou do `.codex/config.toml`; não o inicie em um terminal que também precise de saída humana. A referência MCP detalha cada ferramenta exposta pelo servidor e as diferenças entre leitura e escrita.

`clickupfy mcp setup`

Configura um MCP nomeado no diretório atual. Com terminal interativo, pergunta nome, clientes, perfil, espaço e lista. Use setas e Enter nos seletores; no seletor de clientes, Espaço marca ou desmarca uma opção. Ctrl+C cancela antes da gravação. As listas incluem pastas e listas sem pasta. O workspace é o associado ao perfil escolhido; cadastre o perfil antes com `clickupfy setup`.

PARÂMETRO	TIPO	OBRIGATORIEDADE E PADRÃO
`--name`	texto	Perguntado; obrigatório sem interação.
`--account`	texto	Perguntado; obrigatório sem interação; perfil já cadastrado.
`--space`	ID	Perguntado; obrigatório sem interação; pertence ao workspace do perfil.
`--list`	ID	Perguntado; obrigatório sem interação; pertence ao espaço escolhido.
`--clients`	CSV	Os quatro clientes por padrão: `claude-code`, `codex`, `gemini`, `opencode`.

PARÂMETRO	TIPO	OBRIGATORIEDADE E PADRÃO
`--project`	caminho	Diretório atual.
`--read-only`	booleano	Desativado; quando presente, expõe somente ferramentas de leitura.
`--force`	booleano	Desativado; substitui uma entrada com o mesmo nome.
`--non-interactive`	booleano	Desativado; exige escolhas por flags.

CLIENTE	ARQUIVO LOCAL	SEÇÃO
Claude Code	`.mcp.json`	`mcpServers`
Codex	`.codex/config.toml`	`mcp_servers`
Gemini CLI	`.gemini/settings.json`	`mcpServers`
OpenCode	`opencode.json`, ou `opencode.jsonc` quando existir	`mcp`

Cada entrada inicia `clickupfy mcp serve` com perfil, workspace, espaço e lista fixos. A pasta da lista é acrescentada quando existe. A API key permanece na configuração pessoal; o setup não troca o perfil ativo nem modifica recursos no ClickUp. O binário `clickupfy` precisa estar disponível no `PATH` do cliente.

Exemplos:

```
clickupfy mcp setup
```

```
clickupfy mcp setup --name produto
```

```
clickupfy mcp setup --name suporte --clients claude-code,gemini
```

```
clickupfy mcp setup --name dev --account empresa --space 90070363783 --list
901328052926 --non-interactive
```

```
clickupfy mcp setup --name dev --account empresa --space 90070363783 --list
901328060001 --clients codex,opencode --project ./app --read-only --force --non-
interactive --json
```

Repita com outro nome para manter vários MCPs no mesmo projeto. Nome repetido sem `--force` é recusado. Fora de um TTY, sem as flags obrigatórias, o comando sai com `2`. Um ID fora da hierarquia, nenhuma opção acessível, falha da API ou configuração inválida encerram o comando antes da gravação. Todos os arquivos são analisados antes de escrever; cada substituição é atômica, mas uma falha de disco pode deixar parte dos clientes atualizada. Os demais servidores e ajustes são preservados. O TOML é reserializado e perde comentários; comentários do JSONC do OpenCode são preservados.

O escopo segue as regras de `mcp serve`: Docs continuam no workspace e as permissões da API key continuam sendo as concedidas pelo ClickUp. O setup não cria uma credencial com permissões reduzidas. Em JSON, o `data` traz `{ name, account, workspace, space, list, args, files }`.

Formatos conferidos nas documentações oficiais de Claude Code, Codex, Gemini CLI e OpenCode.

Conecte agentes pelo servidor MCP

Configure pelo terminal

Execute `clickupfy mcp setup` dentro do projeto. O assistente pergunta o nome do MCP e permite selecionar clientes, perfil cadastrado, Space e List. Ele configura Claude Code, Codex, Gemini CLI e OpenCode nos arquivos locais de cada cliente. Repita com outro nome para adicionar mais um MCP.

```
clickupfy mcp setup
```

Sem terminal, informe os valores:

```
clickupfy mcp setup \
  --name promovaweb-clickupfy \
  --account promovaweb \
  --space 10 \
  --list 30 \
  --clients claude-code,codex,gemini,opencode \
  --non-interactive
```

Consulte as flags, arquivos gerados e exemplos na referência de setup.

Inicie manualmente

O servidor usa transporte stdio e reserva a saída padrão para as mensagens JSON-RPC trocadas com o cliente MCP:

```
clickupfy mcp
```

`clickupfy mcp serve` mantém a entrada do ClickUpfy 0.9.0 e aceita a List do projeto:

```
clickupfy mcp serve --list 30
```

A List é obrigatória nessa entrada porque limita consultas e mutações ao destino do projeto. O comando abaixo também fixa os níveis superiores e o Sprint Folder:

```
clickupfy mcp serve \
  --account promovaweb \
  --workspace 123 \
  --space 10 \
  --folder 20 \
  --list 30 \
  --sprint-folder 40
```

Na forma nova, `clickupfy mcp` aceita `--list-id <id>` no lugar de `--list`:

```
clickupfy mcp --list-id 30 --read-only
```

Na prática, deixe o cliente MCP iniciar esse processo pelo `.mcp.json` ou pelo `.codex/config.toml` quando o cliente for o Codex. O comando `agent install` gera e atualiza os dois formatos; `mcp setup` cobre também Gemini e OpenCode.

Ative read-only

```
clickupfy mcp serve --list 30 --read-only
```

O modo read-only não registra ferramentas de escrita. Elas deixam de aparecer em `tools/list`, portanto a restrição é aplicada na superfície do servidor e não depende apenas da obediência do agente.

Use esse modo para descoberta, auditoria, demonstração ou qualquer conversa que não autorize alterações. Confira `tools/list` no cliente para confirmar que as ferramentas de escrita não foram registradas.

Entenda o isolamento

O MCP recebe IDs no processo. Quando uma ferramenta omite um ID, o servidor usa o valor fixado. Quando recebe um valor diferente, recusa a operação com `isError`. Essa comparação protege estes níveis:

- account
- workspace
- Space
- Folder
- List
- Sprint Folder.

A busca de tarefas também permanece dentro da List do projeto. Assim, dois clientes MCP podem operar em paralelo sem compartilhar destino.

Uma falha esperada volta com `isError: true`, a mensagem e o exit code equivalente do CLI. Ferramentas destrutivas exigem `confirm: true`.

Ferramentas de contexto e navegação

`clickupfy_mcp_context` mostra perfil e hierarquia sem expor a API key. `clickupfy_list_get` retorna os status válidos da List.

Catálogo disponível em read-only

FERRAMENTA	FUNÇÃO
`clickupfy_mcp_context`	Mostra o perfil e os IDs fixados pelo projeto.
`clickupfy_accounts_list`	Lista perfis locais sem revelar API keys.
`clickupfy_whoami`	Valida o perfil e retorna o usuário autenticado.
`clickupfy_workspaces_list`	Lista workspaces autorizados.
`clickupfy_spaces_list`	Lista Spaces do workspace do perfil.
`clickupfy_folders_list`	Lista Folders do Space fixado ou informado.
`clickupfy_lists_list`	Lista Lists de um Folder ou diretamente de um Space.
`clickupfy_list_get`	Lê a List do projeto e seus status.
`clickupfy_tasks_list`	Lista tarefas da List autorizada.
`clickupfy_tasks_search`	Busca tarefas dentro da List autorizada.
`clickupfy_task_get`	Lê uma tarefa como resposta original, fila `execution` ou Markdown.
`clickupfy_comments_list`	Lê os comentários de uma tarefa.
`clickupfy_tags_list`	Lê as tags de um Space.
`clickupfy_sprints_list`	Lista Sprints do Sprint Folder.
`clickupfy_sprint_current`	Obtém a Sprint ativa na data de referência.
`clickupfy_sprint_get`	Produz o relatório de uma Sprint.
`clickupfy_sprint_tasks`	Lista tarefas associadas à Sprint.
`clickupfy_time_current`	Consulta o time entry em execução.
`clickupfy_docs_list`	Busca Docs do workspace do perfil.

FERRAMENTA	FUNÇÃO
`clickupfy_doc_get`	Lê metadados de um Doc.
`clickupfy_doc_page_tree`	Lê a hierarquia de páginas do Doc, sem conteúdo.
`clickupfy_doc_pages_list`	Lê as páginas do Doc com conteúdo em Markdown.
`clickupfy_doc_page_get`	Lê uma página específica com conteúdo.

Catálogo acrescentado no modo com escrita

FERRAMENTA	FUNÇÃO
`clickupfy_account_use`	Altera o perfil ativo quando o servidor não fixa um perfil.
`clickupfy_workspace_use`	Altera o workspace quando perfil e workspace não estão fixados.
`clickupfy_task_create`	Cria tarefa ou subtarefa na List autorizada.
`clickupfy_task_update`	Atualiza campos de uma tarefa.
`clickupfy_task_delete`	Exclui uma tarefa mediante `confirm: true`.
`clickupfy_task_add_tag`	Adiciona uma tag sem trocar as outras.
`clickupfy_task_remove_tag`	Remove uma tag e a mantém no Space.
`clickupfy_tag_create`	Cria uma tag no Space.
`clickupfy_tag_update`	Renomeia uma tag ou troca as cores.
`clickupfy_tag_delete`	Exclui uma tag do Space mediante `confirm: true`.
`clickupfy_checklist_create`	Cria um checklist em uma tarefa.
`clickupfy_checklist_item_create`	Acrescenta um item ao checklist.
`clickupfy_checklist_item_set`	Marca ou reabre um item e confirma o novo estado.
`clickupfy_sprint_add_task`	Associa uma tarefa à Sprint.
`clickupfy_sprint_remove_task`	Remove a associação sem excluir a tarefa.
`clickupfy_sprint_set_points`	Define Sprint Points.
`clickupfy_comment_create`	Publica um comentário na tarefa.
`clickupfy_time_start`	Inicia um time entry associado à tarefa.
`clickupfy_time_stop`	Encerra o time entry atual.
`clickupfy_doc_create`	Cria um Doc no workspace, opcionalmente vinculado a um local.
`clickupfy_doc_page_create`	Cria uma página, ou sub-página, em um Doc.

FERRAMENTA	FUNÇÃO
`clickupfy_doc_page_update`	Atualiza título, subtítulo e/ou conteúdo de uma página.

Além dessas, cada operação do catálogo genérico vira uma tool `clickupfy_api_<versão>_<slug>`, com 173 operações da API v2 e v3. Em `--read-only`, só as operações de leitura são registradas.

Na criação e na atualização, informe apenas os campos que devem ser enviados. O servidor recusa IDs que não correspondem ao escopo fixado. A skill de implementação consulta `clickupfy_time_current` antes de iniciar outro registro.

Em `clickupfy_task_create`, datas são timestamp em milissegundos. Em `clickupfy_task_update`, `clearStartDate` e `clearDueDate` removem as datas.

As ferramentas de Docs não seguem o isolamento por Space, Folder ou List: elas sempre operam no workspace do perfil resolvido, porque Docs não pertencem a uma List. `clickupfy_doc_create` exige `parentType` sempre que `parentId` for informado, seguindo os mesmos tipos de local da API do ClickUp (`4` Space, `5` Folder, `6` List, `7` Everything, `12` tarefa).

Use Markdown para contexto longo

`clickupfy_task_get` aceita `markdown: true`. Nesse modo, a ferramenta retorna um texto único, sem envolver o documento em uma string JSON escapada. Essa forma reduz ruído para agentes e preserva checkboxes hierárquicos.

Esta solicitação usa a leitura em Markdown e informa ao agente que nenhuma mutação está autorizada:

```
Leia a tarefa 86abc123 com clickupfy_task_get e markdown: true. Resuma o
objetivo, liste os itens pendentes e não faça mutações.
```

Liste e teste o servidor

```
clickupfy mcp --list
clickupfy mcp --list --json
npx @modelcontextprotocol/inspector clickupfy mcp
```

`--list` descreve tools, resources e prompts sem iniciar o servidor. O Inspector abre uma página local para chamar tools e inspecionar as mensagens JSON-RPC.

Diagnostique a conexão

Se o cliente não listar ferramentas, siga a conexão desde o comando local até o recarregamento do processo:

1. execute o comando de `args` manualmente no terminal
2. confira se o perfil existe
3. confirme que `--list` ou `--list-id` possui valor
4. valide o JSON ou o TOML do cliente
5. rode `clickupfy doctor` e `clickupfy doctor --online`
6. reinicie o cliente MCP
7. leia os logs sem imprimir a API key.

A referência do CLI reúne os comandos e as flags usados para reproduzir o servidor manualmente quando o diagnóstico exigir uma comparação.

Referência MCP: contexto, perfis e hierarquia

Ferramentas de identificação

`clickupfy_mcp_context`

Mostra o perfil resolvido, o workspace associado e a hierarquia fixada pelo processo MCP. Execute-a como primeira chamada de um agente: ela informa quais IDs podem ser omitidos nas ferramentas seguintes, se o servidor é somente leitura e se o projeto tem Sprint Folder fixado.

O `data` traz `{ account, workspace, readOnly, scope }`, e `scope` reúne os IDs fixados. O único argumento aceito é `account`, opcional.

CAMPO	TIPO	OBRIGATÓRIO	EFEITO
`account`	string	não	Perfil a resolver quando o processo não o fixou.

Exemplos:

```
{}
```

```
{"account":"produto"}
```

```
{"account":"cliente-a"}
```

```
{"account":"homologacao"}
```

```
{"account":"suporte"}
```

Uma chamada sem argumento não seleciona outro perfil: ela usa o perfil fixado ou o perfil ativo do `accounts.json`. Guarde os valores de `scope` durante a sessão, porque eles explicam por que uma ferramenta exige ou recusa um ID. Em um servidor fixado, passar outro perfil retorna `isError` e não troca o destino.

`clickupfy_accounts_list`

Lista os perfis configurados na máquina, com nome, usuário, workspace e marca do perfil ativo. Ela não recebe argumentos e não faz requisição à API do ClickUp. Use-a para localizar o identificador de um perfil, nunca para obter uma credencial. O retorno é `{ accounts: [] }`, sem API keys.

Exemplos:

```
{}
```

```
{}
```

```
{}
```

```
{}
```

```
{}
```

Os cinco exemplos são idênticos porque a ferramenta não possui parâmetro. Em um cliente com chamada nomeada, a variação fica no nome da ferramenta, não nos argumentos. Esse fato é importante para agentes: não invente filtros como `workspace`, `query` ou `includeArchived`, pois o schema os rejeita. Como a ferramenta não recebe `account`, ela sempre lê o arquivo local da máquina em que o servidor roda.

`clickupfy_whoami`

Valida o perfil escolhido na API do ClickUp e retorna o usuário autenticado, além do workspace associado ao perfil. É a confirmação remota para uma chave que pode ter sido revogada ou cujas permissões acabaram de mudar. O retorno é `{ account, workspace, user }`.

CAMPO	TIPO	OBRIGATÓRIO	EFEITO
`account`	string	não	Perfil que será autenticado.

Exemplos:

```
{}
```

```
{"account":"produto"}
```

```
{"account":"cliente-a"}
```

```
{"account":"homologacao"}
```

```
{"account":"suporte"}
```

Quando `account` for omitido, o mesmo resolvedor usado por `clickupfy_mcp_context` escolhe o perfil. Uma resposta bem-sucedida prova que a chave funciona naquele momento; não prova permissão para uma List específica. Consulte `clickupfy_mcp_context` e `clickupfy_list_get` para conferir o destino do projeto e os status permitidos. Chave ausente no cofre retorna `isError` com o exit code `4`; chave recusada pela API retorna `5`.

`clickupfy_workspaces_list`

Lista os workspaces autorizados pela API key do perfil. Ela não recebe ID de workspace porque consulta todos os workspaces acessíveis, e o servidor usa o perfil para autenticar a chamada. O retorno é `{ workspaces: [] }`.

CAMPO	TIPO	OBRIGATÓRIO	EFEITO
`account`	string	não	Perfil cuja chave será usada na consulta.

Exemplos:

```
{}
```

```
{"account":"produto"}
```

```
{"account":"cliente-a"}
```

```
{"account":"homologacao"}
```

```
{"account":"suporte"}
```

Essa ferramenta apenas lê os workspaces. Para alterar a associação do perfil, `clickupfy_workspace_use` só fica disponível quando o servidor tem escrita e não foi iniciado com `--account` nem `--workspace`. O valor retornado aqui deve ser usado literalmente nessa ferramenta de escrita ou no CLI.

Ferramentas de navegação

`clickupfy_spaces_list`

Lista Spaces do workspace associado ao perfil. A ferramenta usa o workspace do perfil ou o que foi fixado no processo. `archived` é opcional e, quando verdadeiro, acrescenta Spaces arquivados. O retorno é `{ spaces: [] }`.

CAMPO	TIPO	OBRIGATÓRIO	EFEITO
`account`	string	não	Perfil usado para a consulta.
`archived`	booleano	não	Inclui Spaces arquivados quando vale `true`.

Exemplos:

```
{}
```

```
{"archived":true}
```

```
{"account":"produto"}
```

```
{"account":"produto","archived":true}
```

```
{"account":"cliente-a","archived":false}
```

O valor `false` é diferente de omitir o campo apenas para deixar a intenção explícita no registro do agente; os dois retornam somente recursos ativos. Escolha um `id` da resposta para chamar `clickupfy_folders_list` ou `clickupfy_lists_list`.

`clickupfy_folders_list`

Lista Folders de um Space. Quando o servidor foi iniciado com `--space`, omita `spaceId` para usar o valor fixado. Quando não há Space fixo, o campo é necessário. Um ID diferente do fixado é recusado para preservar o isolamento do projeto. O retorno é `{ folders: [] }`.

CAMPO	TIPO	OBRIGATÓRIO	EFEITO
`account`	string	não	Perfil usado para a consulta.

CAMPO	TIPO	OBRIGATÓRIO	EFEITO
`spaceId`	string	condicional	Space a consultar; obrigatório sem Space fixo.
`archived`	booleano	não	Inclui Folders arquivados.

Exemplos:

```
{}
```

```
{"spaceId":"90070363783"}
```

```
{"spaceId":"90070363783","archived":true}
```

```
{"account":"produto","spaceId":"90070363783"}
```

```
{"account":"cliente-a","spaceId":"9007154145","archived":false}
```

O primeiro exemplo só funciona quando o processo MCP já recebeu `--space`. Uma resposta vazia pode significar que o Space guarda Lists diretamente; ela não autoriza assumir que o projeto não tem Lists. Nesse caso, consulte `clickupfy_lists_list` com `spaceId`.

`clickupfy_lists_list`

Lista Lists de um Folder ou Lists criadas diretamente em um Space. O servidor resolve `folderId` e `spaceId` contra os IDs fixados. Quando um Folder é resolvido, ele tem precedência e o Space não é enviado à API. Sem Folder, sem Space e sem valor fixado, a ferramenta recusa a chamada porque não há nível da hierarquia para consultar. O retorno é `{ lists: [] }`.

CAMPO	TIPO	OBRIGATÓRIO	EFEITO
`account`	string	não	Perfil usado para a consulta.
`folderId`	string	condicional	Folder que contém as Lists.
`spaceId`	string	condicional	Space usado quando as Lists não pertencem a Folder.

CAMPO	TIPO	OBRIGATÓRIO	EFEITO
`archived`	booleano	não	Inclui Lists arquivadas.

Exemplos:

```
{}
```

```
{"folderId":"2001"}
```

```
{"folderId":"2001","archived":true}
```

```
{"spaceId":"90070363783"}
```

```
{"account":"produto","spaceId":"90070363783","archived":false}
```

A primeira chamada exige que o processo tenha iniciado com `--folder` ou `--space`. Não passe IDs de ambos os níveis para tentar ampliar a resposta: o Folder prevalece. A separação permite que o agente descubra a hierarquia sem atravessar o destino do projeto.

`clickupfy_list_get`

Obtém os metadados de uma List e os status aceitos por suas tarefas. A List fixada no processo é a escolha normal: omita `listId` para usá-la. Só informe o campo em um servidor sem List fixa. O retorno é a List da API com `statuses[]`.

CAMPO	TIPO	OBRIGATÓRIO	EFEITO
`account`	string	não	Perfil usado para a consulta.
`listId`	string	condicional	List a obter; omita para usar a List fixada.

Exemplos:

```
{}
```

```
{"listId":"901328052926"}
```

```
{"account":"produto"}
```

```
{"account":"produto","listId":"901328052926"}
```

```
{"account":"cliente-a","listId":"901328060001"}
```

Leia os nomes dos status retornados antes de chamar uma ferramenta que crie ou atualize tarefa. `status` não é um conjunto global do ClickUp: uma grafia como `in review` pode existir em uma List e ser recusada em outra. Esta ferramenta não altera o status de nenhuma tarefa.

Referência MCP: Docs e administração de perfil

Leitura de Docs

`clickupfy_docs_list`

Busca Docs do workspace do perfil. `maxPages` aceita inteiros de `1` a `50`. Quando `parentId` for informado, `parentType` informa o tipo do local: `4` Space, `5` Folder, `6` List, `7` Everything ou `12` tarefa. O retorno é `{ docs: [] }`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`query`	string	não	Nome ou ID do Doc.
`parentId`	string	não	Local pai do Doc.
`parentType`	inteiro	não	Tipo de `parentId`.
`deleted`, `archived`	booleano	não	Inclui Docs excluídos ou arquivados.
`creator`	inteiro	não	ID numérico da pessoa criadora.
`maxPages`	inteiro	não	Cursor de `1` a `50`.

Exemplos:

```
{}
```

```
{"query":"API"}
```

```
{"parentId":"901328052926","parentType":6}
```

```
{"archived":true,"deleted":true,"creator":42,"maxPages":10}
```

```
{"account":"produto","query":"Manual","maxPages":5}
```

Use o ID retornado em `clickupfy_doc_get`, `clickupfy_doc_page_tree` ou uma ferramenta de página. A busca não devolve todo o conteúdo das páginas e não se limita à List fixada pelo MCP. `maxPages` fora de `1` a `50` é recusado pelo schema.

`clickupfy_doc_get`

Obtém metadados de um Doc. O corpo pertence às páginas, por isso esta ferramenta é indicada para confirmar nome, local e estado do Doc antes de consultar ou alterar uma página.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`docId`	string	sim	Doc a obter.

Exemplos:

```
{"docId":"8cdu8-123"}
```

```
{"docId":"8cdu8-456"}
```

```
{"account":"produto","docId":"8cdu8-123"}
```

```
{"account":"cliente-a","docId":"8cdu8-456"}
```

```
{"account":"homologacao","docId":"8cdu8-789"}
```

Se o Doc não pertencer ao workspace do perfil, a API recusa a chamada. Não substitua `docId` por `pageId`: os dois identificadores pertencem a recursos distintos. Doc inexistente retorna `isError` com o exit code `6`.

`clickupfy_doc_page_tree`

Retorna a hierarquia de páginas de um Doc sem trazer os conteúdos, em `{ pages: [] }` aninhadas. É a escolha adequada para localizar `pageId`, parentesco e profundidade antes de criar uma subpágina ou atualizar uma página existente.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`docId`	string	sim	Doc cuja árvore será lida.

Exemplos:

```
{"docId":"8cdu8-123"}
```

```
{"docId":"8cdu8-456"}
```

```
{"account":"produto","docId":"8cdu8-123"}
```

```
{"account":"cliente-a","docId":"8cdu8-456"}
```

```
{"account":"homologacao","docId":"8cdu8-789"}
```

Essa ferramenta não é uma cópia de segurança do texto. Use `clickupfy_doc_pages_list` para ler muitas páginas ou `clickupfy_doc_page_get` para ler uma página conhecida.

`clickupfy_doc_pages_list`

Lista páginas de um Doc com conteúdo. `maxPageDepth` limita o retorno de subpáginas. `contentFormat` aceita `text/md` ou `text/plain`; quando omitido, a API usa `text/md`. O retorno é `{ pages: [] }` com conteúdo.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`docId`	string	sim	Doc cujas páginas serão lidas.
`maxPageDepth`	inteiro	não	Profundidade máxima de subpáginas.
`contentFormat`	string	não	Forma do conteúdo, `text/md` ou `text/plain`.

Exemplos:

```
{"docId":"8cdu8-123"}
```

```
{"docId":"8cdu8-123","maxPageDepth":1}
```

```
{"docId":"8cdu8-123","maxPageDepth":3}
```

```
{"docId":"8cdu8-123","contentFormat":"text/plain"}
```

```
{"account":"produto","docId":"8cdu8-123","contentFormat":"text/md"}
```

Leia somente a profundidade necessária para reduzir conteúdo irrelevante na conversa do agente. A ferramenta não modifica o Doc nem a árvore.

`clickupfy_doc_page_get`

Obtém uma página específica e seu conteúdo. O `pageId` deve pertencer ao Doc informado; localize-o pela árvore ou pela listagem de páginas.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`docId`	string	sim	Doc proprietário.
`pageId`	string	sim	Página a obter.
`contentFormat`	string	não	Forma do conteúdo.

Exemplos:

```
{"docId":"8cdu8-123","pageId":"8cdu8-456"}
```

```
{"docId":"8cdu8-123","pageId":"8cdu8-456","contentFormat":"text/plain"}
```

```
{"docId":"8cdu8-123","pageId":"8cdu8-789","contentFormat":"text/md"}
```

```
{"account":"produto","docId":"8cdu8-123","pageId":"8cdu8-456"}
```

```
{"account":"cliente-a","docId":"8cdu8-456","pageId":"8cdu8-999"}
```

Escrita de Docs

As três ferramentas seguintes não aparecem em `--read-only`. A API pública usada pelo ClickUpfy não expõe ferramentas para excluir Docs ou páginas, mudar permissões de um Doc existente ou reordenar a árvore depois da criação.

`clickupfy_doc_create`

Cria um Doc no workspace. `name` é obrigatório. Quando `parentId` é informado, `parentType` também é obrigatório; `parentId` sem `parentType` é recusado antes de tocar na API. `visibility` pode ser `PRIVATE` ou `PUBLIC`; `createPage` cria uma página em branco junto do Doc.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`name`	string	sim	Nome do Doc.
`parentId`	string	não	Local pai.
`parentType`	inteiro	condicional	Tipo do local pai.
`visibility`	string	não	Visibilidade do novo Doc.
`createPage`	booleano	não	Cria a primeira página vazia.

Exemplos:

```
{"name":"Manual da API"}
```

```
{"name":"Guia do produto","createPage":true}
```

```
{"name":"Notas da List","parentId":"901328052926","parentType":6}
```

```
{"name":"Documento interno","parentId":"2001","parentType":5,"visibility":"PRIVATE"}
```

```
{"account":"produto","name":"Plano de
release","parentId":"86abc123","parentType":12,"createPage":true}
```

Chame `clickupfy_doc_get` depois da criação e use o ID devolvido para criar as páginas. Criação sem `name` é recusada pelo schema.

`clickupfy_doc_page_create`

Cria página ou subpágina em um Doc. `name` é obrigatório. `parentPageId` define o pai; `orderindex` indica a posição entre páginas irmãs. O conteúdo e o subtítulo são opcionais.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`, `docId`	string	`docId` sim	Perfil e Doc proprietário.
`name`	string	sim	Título.

CAMPO	TIPO	OBRIGATÓRIO	USO
`content`, `subTitle`	string	não	Corpo e subtítulo iniciais.
`parentPageId`	string	não	Página pai.
`orderindex`	número	não	Posição entre páginas irmãs.
`contentFormat`	string	não	Forma do conteúdo.

Exemplos:

```
{"docId":"8cdu8-123","name":"Introdução"}
```

```
{"docId":"8cdu8-123","name":"Autenticação","content":"Use uma API key pessoal."}
```

```
{"docId":"8cdu8-123","name":"Detalhes","subTitle":"Campos e
respostas","content":"## Parâmetros"}
```

```
{"docId":"8cdu8-123","name":"Erros","parentPageId":"8cdu8-456","orderindex":2}
```

```
{"account":"produto","docId":"8cdu8-123","name":"Integração","content":"#
MCP","contentFormat":"text/md"}
```

Leia a página criada com `clickupfy_doc_page_get` e a árvore com `clickupfy_doc_page_tree` para confirmar conteúdo e posição.

`clickupfy_doc_page_update`

Atualiza título, subtítulo e/ou conteúdo. O schema recusa chamada sem campo de alteração. `contentEditMode` aceita `replace`, `append` ou `prepend`; o padrão é `replace`. O modo afeta somente `content`, não o título ou subtítulo.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`, `docId`, `pageId`	string	Doc e página sim	Recurso a alterar.
`name`, `subTitle`, `content`	string	ao menos um	Campos a persistir.
`contentEditMode`	enum	não	`replace`, `append` ou `prepend`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`contentFormat`	string	não	Forma do conteúdo.

Exemplos:

```
{"docId":"8cdu8-123","pageId":"8cdu8-456","name":"Visão geral"}
```

```
{"docId":"8cdu8-123","pageId":"8cdu8-456","subTitle":"Instalação e configuração"}
```

```
{"docId":"8cdu8-123","pageId":"8cdu8-456","content":"# Novo conteúdo"}
```

```
{"docId":"8cdu8-123","pageId":"8cdu8-456","content":"\n##
Changelog","contentEditMode":"append"}
```

```
{"account":"produto","docId":"8cdu8-123","pageId":"8cdu8-789","content":"#
Aviso\n\n","contentEditMode":"prepend","contentFormat":"text/md"}
```

Leia a página com `clickupfy_doc_page_get` após atualizar. Não envie um objeto vazio para testar a conexão: use uma das ferramentas de leitura, pois uma atualização sem conteúdo é recusada.

Administração condicionada ao servidor

`clickupfy_account_use`

Define o perfil ativo local. Só aparece quando o servidor não recebeu `--account` e não está em `--read-only`. É a equivalência MCP de `clickupfy account use`; em projetos isolados, prefira fixar o perfil na configuração do processo para evitar troca global durante o trabalho.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	sim	Perfil local já configurado.

Exemplos:

```
{"account":"produto"}
```

```
{"account":"cliente-a"}
```

```
{"account":"homologacao"}
```

```
{"account":"suporte"}
```

```
{"account":"desenvolvimento"}
```

O retorno informa `{ activeAccount }`. Esta ação muda o `accounts.json` da máquina, logo não deve ser usada para uma consulta pontual de outro perfil. Perfil inexistente retorna `isError` com o exit code `6`.

`clickupfy_workspace_use`

Associa um workspace autorizado ao perfil. Só aparece quando o processo não fixou `--account` nem `--workspace` e tem escrita. O servidor confere a autorização contra a API antes de salvar a associação no perfil local.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a alterar; quando omitido usa o ativo.
`workspaceId`	string	sim	Workspace autorizado para o perfil.

Exemplos:

```
{"workspaceId":"90070363783"}
```

```
{"account":"produto","workspaceId":"90070363783"}
```

```
{"account":"cliente-a","workspaceId":"9007154145"}
```

```
{"account":"homologacao","workspaceId":"90070363783"}
```

```
{"account":"suporte","workspaceId":"9007154145"}
```

O retorno é `{ account, workspace }`. Use `clickupfy_workspaces_list` para descobrir um ID autorizado. Uma tentativa com workspace ausente da lista é recusada com `isError` e não altera a associação existente.

Referência MCP: tarefas, Sprints, checklists, comentários e tempo

Tarefas: leitura e busca

`clickupfy_tasks_list`

Lista tarefas da List autorizada. Omitir `listId` usa a List fixada. Os filtros aceitam arrays: `status` é uma lista de nomes configurados na List e `assignees` contém IDs de pessoas em texto. `page` deve ser inteiro maior ou igual a zero.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`listId`	string	não	List fixa do projeto quando omitido.
`status`	string[]	não	Filtra por um ou mais status.
`assignees`	string[]	não	Filtra por responsáveis.
`includeClosed`	booleano	não	Inclui tarefas concluídas.
`page`	inteiro	não	Página, iniciando em zero.

Exemplos:

```
{}
```

```
{"status":["in progress"]}
```

```
{"assignees":["42","57"],"includeClosed":false}
```

```
{"includeClosed":true,"page":1}
```

```
{"account":"produto","listId":"901328052926","status":["to do","in progress"]}
```

Cada página traz até 100 tarefas. Para obter descrição, subtarefas e checklist items de uma tarefa, chame `clickupfy_task_get` com o `id` devolvido. Nunca use um `listId` diferente do fixado pelo projeto.

`clickupfy_task_get`

Obtém a tarefa e acrescenta `execution`, uma fila endereçável que preserva subtarefas e checklist items. `raw` devolve somente a resposta original da API. `markdown` devolve um texto único com descrição e itens; os dois campos são mutuamente exclusivos e, juntos, produzem erro.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`taskId`	string	sim	Tarefa raiz.
`raw`	booleano	não	Retorna somente o payload do ClickUp.
`markdown`	booleano	não	Renderiza tarefa e fila em Markdown.

Exemplos:

```
{"taskId":"86abc123"}
```

```
{"taskId":"86abc123","markdown":true}
```

```
{"taskId":"86abc123","raw":true}
```

```
{"account":"produto","taskId":"86abc123"}
```

```
{"account":"cliente-a","taskId":"77def456","markdown":true}
```

Os itens usam chaves como `task:86abc123` e `checklist:b955c4dc:21e08dc8`. `parentKey` e `depth` preservam a árvore, e `state` vale `done` ou `pending`. A ação `complete` de cada item mostra o comando do CLI e a chamada MCP adequados, mas a autorização para executar essa ação continua vindo do pedido recebido pelo agente.

`clickupfy_tasks_search`

Busca tarefas por texto, status e responsável. Com List fixada, percorre somente essa List, página por página; sem List fixada, percorre o workspace, como o CLI. `maxPages` aceita inteiro entre `1` e `100`. Omitir `query` lista de acordo com os outros filtros, portanto inclua um termo quando a intenção for uma busca nominal.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.

CAMPO	TIPO	OBRIGATÓRIO	USO
`query`	string	não	Texto no ID, nome ou descrição.
`status`	string[]	não	Filtra por status.
`assignees`	string[]	não	Filtra por responsáveis.
`includeClosed`	booleano	não	Inclui tarefas concluídas.
`maxPages`	inteiro	não	Paginação, de `1` a `100`.

Exemplos:

```
{"query":"autenticação"}
```

```
{"query":"86abc123"}
```

```
{"query":"API","status":["in progress"]}
```

```
{"assignees":["42"],"includeClosed":true,"maxPages":5}
```

```
{"account":"produto","query":"checkout","maxPages":10}
```

Se duas tarefas forem plausíveis, apresente ID e nome e aguarde a escolha da pessoa que solicitou o trabalho. A ferramenta não altera tarefa e não expande a busca para uma List não autorizada.

`clickupfy_comments_list`

Lê os comentários de uma tarefa, com todas as páginas da API. O retorno normaliza usuário, data e texto para facilitar a leitura pelo agente, em `{ comments: [] }` com `id`, `user`, `date` e `text`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`taskId`	string	sim	Tarefa cujos comentários serão lidos.

Exemplos:

```
{"taskId":"86abc123"}
```

```
{"taskId":"77def456"}
```

```
{"account":"produto","taskId":"86abc123"}
```

```
{"account":"cliente-a","taskId":"77def456"}
```

```
{"account":"homologacao","taskId":"55ghi789"}
```

Leia essa ferramenta antes de publicar mudança de status ou comentário de progresso, e não trate um comentário antigo como instrução mais recente sem conferir as datas. Ela não expõe anexos como conteúdo, não publica respostas e não altera o campo de status.

Sprints

`clickupfy_sprints_list`

Lista Sprints dentro do Sprint Folder fixado. Uma Sprint é uma List que possui `start_date` e `due_date`; `includeRegular` acrescenta Lists comuns ao retorno de diagnóstico. `at` recebe uma data `AAAA-MM-DD` usada para classificar o estado do ciclo.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`folderId`	string	não	Sprint Folder fixado quando omitido.
`archived`	booleano	não	Inclui Lists arquivadas.
`includeRegular`	booleano	não	Inclui Lists que não são Sprints.
`at`	string	não	Data de referência em `AAAA-MM-DD`.

Exemplos:

```
{}
```

```
{"at":"2026-11-13"}
```

```
{"archived":true,"includeRegular":true}
```

```
{"folderId":"4001","at":"2026-12-01"}
```

```
{"account":"produto","folderId":"4001","includeRegular":false}
```

O objeto vazio funciona somente se o servidor recebeu `--sprint-folder`; sem Sprint Folder fixado, `folderId` é obrigatório. O retorno é `{ sprints: [] }` com `id`, `name`, `startDate`, `dueDate`, `status`, `taskCount` e `isSprint`. A ferramenta não cria Sprint nem altera datas: a API pública do ClickUp não oferece criação de Sprint por esse fluxo.

`clickupfy_sprint_current`

Obtém a única Sprint cujo período contém a data informada ou a data atual. Se não houver Sprint ativa, a ferramenta retorna erro com o exit code `6`; se dois períodos se sobrepuserem, retorna erro com as candidatas em vez de escolher uma List arbitrariamente.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`folderId`	string	não	Sprint Folder fixado quando omitido.
`at`	string	não	Data de referência em `AAAA-MM-DD`.

Exemplos:

```
{}
```

```
{"at":"2026-11-13"}
```

```
{"folderId":"4001"}
```

```
{"folderId":"4001","at":"2026-12-01"}
```

```
{"account":"produto","folderId":"4001","at":"2026-10-15"}
```

Não use a falha como motivo para anexar tarefa a qualquer List do Folder. Corrija datas e sobreposição no ClickUp, depois leia novamente a Sprint atual.

`clickupfy_sprint_get`

Produz o relatório de uma Sprint. Inclui período, progresso por quantidade de tarefas, progresso por Sprint Points e distribuição de status. `at` muda somente o cálculo temporal da situação da Sprint.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`sprintId`	string	sim	Sprint a analisar.
`at`	string	não	Data de referência em `AAAA-MM-DD`.

Exemplos:

```
{"sprintId":"901328052926"}
```

```
{"sprintId":"901328052926","at":"2026-11-13"}
```

```
{"sprintId":"901328060001"}
```

```
{"account":"produto","sprintId":"901328052926"}
```

```
{"account":"cliente-a","sprintId":"901328060002","at":"2026-12-01"}
```

O retorno tem `totalTasks`, `completedTasks`, `openTasks`, `totalPoints`, `completedPoints`, `unestimatedTasks`, `taskCompletionPercent`, `pointsCompletionPercent` e `tasksByStatus`. Uma tarefa sem Points participa do total de tarefas, mas não adiciona peso ao cálculo por pontos. Leia `clickupfy_sprint_tasks` para obter os itens do relatório, não tente inferi-los apenas pelas porcentagens.

`clickupfy_sprint_tasks`

Lista tarefas associadas a uma Sprint, incluindo concluídas por padrão. `openOnly: true` remove tarefas já concluídas e é útil para a fila presente, mas não substitui a lista integral usada na revisão final do ciclo.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`sprintId`	string	sim	Sprint consultada.

CAMPO	TIPO	OBRIGATÓRIO	USO
`openOnly`	booleano	não	Omite tarefas concluídas quando vale `true`.

Exemplos:

```
{"sprintId":"901328052926"}
```

```
{"sprintId":"901328052926","openOnly":true}
```

```
{"sprintId":"901328060001"}
```

```
{"account":"produto","sprintId":"901328052926","openOnly":false}
```

```
{"account":"cliente-a","sprintId":"901328060002","openOnly":true}
```

A ferramenta retorna `{ tasks: [] }` em linhas compactas. Para descrição, subtarefas e checklist items de um item específico, chame `clickupfy_task_get` com o ID da tarefa.

Ferramentas de escrita de trabalho

As ferramentas seguintes só aparecem fora de `--read-only`. Informe apenas campos que devem mudar. Elas usam o mesmo perfil e as mesmas regras de isolamento das consultas.

`clickupfy_task_create`

Cria tarefa ou subtarefa na List fixa. `name` é obrigatório. Datas devem ser timestamps inteiros em milissegundos, diferentemente do CLI, que aceita datas `AAAA-MM-DD` gravadas no começo do dia local.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`, `listId`	string	não	Perfil e List fixa quando omitida.
`name`	string	sim	Nome da tarefa.
`description`, `markdownContent`, `status`, `parent`	string	não	Conteúdo, status e pai.
`priority`	inteiro	não	De `1` a `4`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`assignees`	string[]	não	IDs numéricos dos responsáveis, em texto.
`startDate`, `dueDate`	inteiro	não	Timestamp em milissegundos.
`points`	número	não	Sprint Points não negativos.

Exemplos:

```
{"name":"Corrigir retorno da API"}
```

```
{"name":"Documentar login","description":"Explicar a sessão."}
```

```
{"name":"Criar testes","markdownContent":"## Casos\n\n- Login válido"}
```

```
{"name":"Implementar token","status":"in progress","priority":2,"assignees":["42","57"],"startDate":1763251200000,"dueDate":1763596800000,"points":5}
```

```
{"account":"produto","listId":"901328052926","name":"Cobrir expiração","parent":"86abc123","markdownContent":"Adicionar testes de sessão expirada."}
```

Leia a tarefa criada com `clickupfy_task_get`. Para `listId`, omitir é o caminho normal no MCP do projeto; um valor diferente do fixo é recusado.

`clickupfy_task_update`

Atualiza campos de uma tarefa. `taskId` é obrigatório e pelo menos um campo de alteração deve estar presente. Para remover uma data, use `clearStartDate: true` ou `clearDueDate: true`; não envie timestamp nulo nem texto vazio.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`, `taskId`	string	`taskId` sim	Perfil e tarefa.
`name`, `description`, `markdownContent`, `status`	string	não	Campos textuais.
`priority`	inteiro	não	De `1` a `4`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`startDate`, `dueDate`	inteiro	não	Nova data em milissegundos.
`clearStartDate`, `clearDueDate`	booleano	não	Remove a data correspondente.
`points`	número	não	Sprint Points não negativos.

Exemplos:

```
{"taskId":"86abc123","status":"in progress"}
```

```
{"taskId":"86abc123","name":"Corrigir autenticação por token"}
```

```
{"taskId":"86abc123","priority":1,"points":8}
```

```
{"taskId":"86abc123","startDate":1763251200000,"dueDate":1763596800000}
```

```
{"account":"produto","taskId":"86abc123","markdownContent":"## Entrega\n\nPublicar
manual atualizado.","clearDueDate":true}
```

Enviar `startDate` junto de `clearStartDate`, ou `dueDate` junto de `clearDueDate`, produz erro. Depois da escrita, use `clickupfy_task_get`. A ferramenta devolve a tarefa atualizada, mas a releitura é importante quando outras alterações concorrem na mesma tarefa.

`clickupfy_checklist_create`

Cria checklist em uma tarefa. O retorno é `{ checklist }` e fornece o ID necessário em `clickupfy_checklist_item_create`. Itens não são criados automaticamente.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`taskId`	string	sim	Tarefa proprietária.
`name`	string	sim	Nome do checklist.

Exemplos:

```
{"taskId":"86abc123","name":"Testes"}
```

```
{"taskId":"86abc123","name":"Revisão de segurança"}
```

```
{"taskId":"77def456","name":"Publicação"}
```

```
{"account":"produto","taskId":"86abc123","name":"Aceite"}
```

```
{"account":"cliente-a","taskId":"77def456","name":"Regressão"}
```

`clickupfy_checklist_item_create`

Acrescenta um item aberto a um checklist. `assignee`, quando informado, é um ID numérico. A ferramenta não recebe `taskId` porque o checklist já define a tarefa proprietária. O retorno é `{ checklist }` com `items[]`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`checklistId`	string	sim	Checklist que receberá o item.
`name`	string	sim	Texto do item.
`assignee`	inteiro	não	ID numérico da pessoa responsável.

Exemplos:

```
{"checklistId":"b955c4dc","name":"Executar testes unitários"}
```

```
{"checklistId":"b955c4dc","name":"Executar build"}
```

```
{"checklistId":"b955c4dc","name":"Revisar documentação","assignee":42}
```

```
{"account":"produto","checklistId":"3f1a7c22","name":"Conferir changelog"}
```

```
{"account":"cliente-a","checklistId":"3f1a7c22","name":"Validar
publicação","assignee":57}
```

`clickupfy_checklist_item_set`

Marca ou reabre um item e confirma o estado por uma nova leitura da tarefa. `taskId` deve ser a tarefa raiz devolvida por `clickupfy_task_get`, `resolved: true` conclui e `resolved: false` reabre.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`taskId`	string	sim	Tarefa raiz usada na conferência.
`checklistId`	string	sim	Checklist proprietário.
`itemId`	string	sim	Item a atualizar.
`resolved`	booleano	sim	`true` conclui; `false` reabre.

Exemplos:

```
{"taskId":"86abc123","checklistId":"b955c4dc","itemId":"21e08dc8","resolved":true}
```

```
{"taskId":"86abc123","checklistId":"b955c4dc","itemId":"21e08dc8","resolved":false}
```

```
{"account":"produto","taskId":"86abc123","checklistId":"3f1a7c22","itemId":"8b2d7e10","resolved"
```

```
{"account":"cliente-
a","taskId":"77def456","checklistId":"3f1a7c22","itemId":"8b2d7e10","resolved":true}
```

```
{"taskId":"77def456","checklistId":"3f1a7c22","itemId":"8b2d7e10","resolved":false}
```

O retorno é o item confirmado na nova leitura, com `done`, `state` e `action`. Não passe um item de outra tarefa: a ferramenta faz essa conferência e recusa a escrita com o exit code `6` quando a relação não corresponde à árvore lida. Se o ClickUp não confirmar o estado, a resposta traz `isError` com o exit code `5`.

`clickupfy_sprint_add_task`

Associa tarefa a Sprint sem trocar sua List principal. Ela não cria tarefa e não muda seu status. A ferramenta só aparece em servidor com escrita. O retorno é `{ sprintId, taskId, added }`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.

CAMPO	TIPO	OBRIGATÓRIO	USO
`sprintId`	string	sim	Sprint de destino.
`taskId`	string	sim	Tarefa a associar.

Exemplos:

```
{"sprintId":"901328052926","taskId":"86abc123"}
```

```
{"sprintId":"901328052926","taskId":"77def456"}
```

```
{"account":"produto","sprintId":"901328052926","taskId":"86abc123"}
```

```
{"account":"cliente-a","sprintId":"901328060002","taskId":"55ghi789"}
```

```
{"account":"homologacao","sprintId":"901328060003","taskId":"99jkl012"}
```

Leia `clickupfy_sprint_tasks` depois da escrita. Só associe a tarefa quando o planejamento da Sprint estiver incluído no pedido.

`clickupfy_sprint_remove_task`

Remove uma associação com Sprint sem excluir a tarefa. Comentários, checklist, tempo e List principal permanecem na tarefa. O retorno é `{ sprintId, taskId, removed }`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`sprintId`	string	sim	Sprint cuja associação será removida.
`taskId`	string	sim	Tarefa associada.

Exemplos:

```
{"sprintId":"901328052926","taskId":"86abc123"}
```

```
{"sprintId":"901328052926","taskId":"77def456"}
```

```
{"account":"produto","sprintId":"901328052926","taskId":"86abc123"}
```

```
{"account":"cliente-a","sprintId":"901328060002","taskId":"55ghi789"}
```

```
{"account":"homologacao","sprintId":"901328060003","taskId":"99jkl012"}
```

Use `clickupfy_task_delete` somente quando a exclusão inteira estiver explicitamente autorizada.

`clickupfy_sprint_set_points`

Define Sprint Points de uma tarefa. `points` é um número maior ou igual a zero. O ClickUpfy transmite o valor; a escala pertence ao time. O retorno é a tarefa atualizada.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`taskId`	string	sim	Tarefa que receberá pontos.
`points`	número	sim	Valor não negativo.

Exemplos:

```
{"taskId":"86abc123","points":0}
```

```
{"taskId":"86abc123","points":1}
```

```
{"taskId":"86abc123","points":3}
```

```
{"account":"produto","taskId":"86abc123","points":5}
```

```
{"account":"cliente-a","taskId":"77def456","points":8}
```

Pontos negativos retornam `isError` sem chamar a API. Leia a tarefa com `clickupfy_task_get` para confirmar o número persistido.

`clickupfy_task_delete`

Exclui permanentemente uma tarefa. O campo `confirm` só aceita o literal `true`; essa exigência impede uma exclusão causada por objeto incompleto ou ambíguo. A ferramenta não está disponível em `--read-only`. O retorno é `{ deleted, taskId }`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`taskId`	string	sim	Tarefa a excluir.
`confirm`	`true`	sim	Confirmação literal obrigatória.

Exemplos:

```
{"taskId":"86abc123","confirm":true}
```

```
{"taskId":"77def456","confirm":true}
```

```
{"account":"produto","taskId":"86abc123","confirm":true}
```

```
{"account":"cliente-a","taskId":"77def456","confirm":true}
```

```
{"account":"homologacao","taskId":"55ghi789","confirm":true}
```

Leia tarefa, nome e List e obtenha autorização explícita para exclusão antes de qualquer exemplo deste tipo. Para somente retirar a tarefa do ciclo, use `clickupfy_sprint_remove_task`.

`clickupfy_comment_create`

Publica comentário em uma tarefa. `notifyAll` é opcional; ele pede ao ClickUp para notificar participantes. A ferramenta não modifica status, timer ou checklist. O retorno é `{ id, hist_id, date }`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`taskId`	string	sim	Tarefa que receberá o comentário.
`text`	string	sim	Conteúdo do comentário.
`notifyAll`	booleano	não	Pede notificação aos participantes.

Exemplos:

```
{"taskId":"86abc123","text":"Iniciei a análise da tarefa."}
```

```
{"taskId":"86abc123","text":"Os testes unitários passaram."}
```

```
{"taskId":"86abc123","text":"Aguardando acesso ao ambiente de
homologação.","notifyAll":true}
```

```
{"account":"produto","taskId":"86abc123","text":"Documentação e ebook foram
atualizados."}
```

```
{"account":"cliente-a","taskId":"77def456","text":"Validação final
concluída.","notifyAll":true}
```

`clickupfy_time_current`

Consulta o time entry que está em execução no workspace. Não recebe filtros; omitir `account` usa o perfil fixado ou ativo. O retorno é `{ entry }`, com `entry: null` quando não há cronômetro.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil cujo workspace será consultado.

Exemplos:

```
{}
```

```
{"account":"produto"}
```

```
{"account":"cliente-a"}
```

```
{"account":"homologacao"}
```

```
{"account":"suporte"}
```

`clickupfy_time_start`

Inicia um time entry para uma tarefa. Consulte `clickupfy_time_current` antes. O ClickUp mantém um cronômetro por pessoa: quando um já roda em outra tarefa, ele é substituído, e a tarefa anterior volta em `previous` com `id`, `taskId`, `taskName`, `description` e `start`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`taskId`	string	sim	Tarefa associada ao tempo.
`description`	string	não	Descrição curta do trabalho.

Exemplos:

```
{"taskId":"86abc123"}
```

```
{"taskId":"86abc123","description":"Implementação"}
```

```
{"taskId":"86abc123","description":"Testes de regressão"}
```

```
{"account":"produto","taskId":"86abc123","description":"Revisão de documentação"}
```

```
{"account":"cliente-a","taskId":"77def456","description":"Correção da integração"}
```

O retorno é `{ entry, previous? }`.

`clickupfy_time_stop`

Encerra o time entry atual do workspace. Ela não recebe `taskId` e não modifica status nem comentários. Releia `clickupfy_time_current` depois da chamada quando o encerramento precisa ser confirmado. Sem cronômetro ativo, a resposta traz `isError` com o exit code `5`.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil cujo registro atual será encerrado.

Exemplos:

```
{}
```

```
{"account":"produto"}
```

```
{"account":"cliente-a"}
```

```
{"account":"homologacao"}
```

```
{"account":"suporte"}
```

O retorno é `{ entry }` com o time entry encerrado.

Tags de tarefas

`clickupfy_task_add_tag`

Adiciona uma tag a uma tarefa ou subtarefa existente, preservando as outras tags. A ferramenta está disponível somente quando o servidor permite escrita. Ela usa a API key do perfil selecionado e respeita o perfil e o workspace fixados pelo MCP do projeto.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO E USO
`taskId`	string	sim	ID interno da tarefa; não aceita texto vazio.
`tagName`	string	sim	Nome da tag; não aceita texto vazio ou composto apenas por espaços.
`customTaskId`	booleano	não	`false`; habilite para resolver o ID no workspace do perfil.
`account`	string	não	Perfil ativo ou fixado no projeto.

Cada chamada associa um nome e retorna `{ added, taskId, tagName }` quando a API aceita a operação. A permissão para alterar a tarefa depende da API key. Recusas do ClickUp retornam `isError: true`; argumentos inválidos são recusados pelo schema. Em `--read-only`, a ferramenta não é registrada. A ferramenta não gerencia as cores ou o catálogo de tags do Space.

Exemplos:

Associe uma tag pelo ID interno:

```
{"taskId":"86abc123","tagName":"bug"}
```

Use um nome com espaços e acentos:

```
{"taskId":"86abc123","tagName":"em revisão"}
```

Resolva um ID personalizado no workspace configurado:

```
{"taskId":"DEV-42","tagName":"backend","customTaskId":true}
```

Selecione outro perfil quando o servidor permitir:

```
{"account":"produto","taskId":"77def456","tagName":"documentação"}
```

Associe uma tag a uma subtarefa pelo ID interno:

```
{"taskId":"sub-1","tagName":"QA / revisão","customTaskId":false}
```

Depois da escrita, consulte `clickupfy_task_get` com `raw: true` e o ID interno para conferir o campo `tags`. IDs personalizados usam `custom_task_ids=true` e `team_id` no endpoint Add Tag To Task.

Gestão do catálogo de tags

As operações usam o perfil ativo ou `account` no MCP. A API key precisa ter acesso ao Space ou à tarefa. Falhas do ClickUp são propagadas e retornam `isError: true`. Ferramentas de escrita não ficam disponíveis em modo somente leitura. O MCP aceita `spaceId` ou usa o Space fixado na configuração; no CLI, o equivalente é `--space`, obrigatório nos comandos `tag`.

`clickupfy_tags_list`

Lista nomes e cores das tags de um Space, em `{ tags: [] }`. A consulta não altera tags.

`spaceId` é string, opcional quando o MCP fixa o Space.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`spaceId`	string	condicional	Space consultado; obrigatório sem Space fixo.

Exemplos:

```
{"spaceId":"90070363783"}
```

```
{"spaceId":"9007154145"}
```

```
{"spaceId":"90070363783","account":"dev"}
```

```
{"spaceId":"9007154145","account":"produto"}
```

```
{}
```

O último exemplo funciona apenas com Space fixado no servidor. Após uma escrita, consulte `clickupfy_tags_list` para conferir o catálogo ou `clickupfy_task_get` com `raw: true` para conferir as associações da tarefa.

`clickupfy_tag_create`

Cria uma tag no catálogo do Space com nome e cores. Retorna `{ created, spaceId, name }`. A associação a tarefas é feita separadamente com `clickupfy_task_add_tag`.

`name`, `foreground` e `background` são obrigatórios. As cores usam `#RRGGBB`. Nome vazio e cor fora do formato são recusados.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`spaceId`	string	condicional	Space do catálogo.
`name`	string	sim	Nome da tag.
`foreground`	string	sim	Cor do texto em `#RRGGBB`.
`background`	string	sim	Cor de fundo em `#RRGGBB`.

Exemplos:

```
{"spaceId":"90070363783","name":"bug","foreground":"#ffffff","background":"#112233"}
```

```
{"spaceId":"90070363783","name":"em
revisão","foreground":"#ffffff","background":"#336699"}
```

```
{"spaceId":"90070363783","name":"backend","foreground":"#ffffff","background":"#112233"}
```

```
{"spaceId":"9007154145","name":"documentação","foreground":"#ffffff","background":"#336699"}
```

```
{"spaceId":"90070363783","name":"QA /
revisão","foreground":"#ffffff","background":"#112233"}
```

`clickupfy_tag_update`

Renomeia a tag ou altera suas cores no Space. Consulta a definição atual e preserva os campos omitidos. Retorna `{ updated, spaceId, name }`. Tag ausente e atualização sem campos são recusadas.

`tagName` é o nome atual obrigatório. `name`, `foreground` e `background` são opcionais, com pelo menos um campo informado.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`spaceId`	string	condicional	Space do catálogo.
`tagName`	string	sim	Nome atual da tag.
`name`	string	não	Novo nome.
`foreground`, `background`	string	não	Novas cores em `#RRGGBB`.

Exemplos:

```
{"spaceId":"90070363783","tagName":"bug","name":"defeito"}
```

```
{"spaceId":"90070363783","tagName":"bug","foreground":"#ffffff"}
```

```
{"spaceId":"90070363783","tagName":"bug","background":"#cc0000"}
```

```
{"spaceId":"90070363783","tagName":"em
revisão","name":"revisada","background":"#00aa00"}
```

```
{"spaceId":"90070363783","tagName":"backend","foreground":"#ffffff","background":"#112233"}
```

`clickupfy_tag_delete`

Exclui a tag do Space e de todas as tarefas que a usavam. Consulte as tarefas que a utilizam antes de executar essa alteração. Retorna `{ deleted, spaceId, name }`. Para remover somente a associação a uma tarefa, use `clickupfy_task_remove_tag`.

`tagName` é obrigatório e `confirm: true` também. O cliente consulta a definição da tag para enviar o corpo DELETE exigido pela API.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`spaceId`	string	condicional	Space do catálogo.
`tagName`	string	sim	Nome atual da tag.
`confirm`	`true`	sim	Confirmação literal obrigatória.

Exemplos:

```
{"spaceId":"90070363783","tagName":"bug","confirm":true}
```

```
{"spaceId":"90070363783","tagName":"em revisão","confirm":true}
```

```
{"spaceId":"90070363783","tagName":"backend","confirm":true}
```

```
{"spaceId":"9007154145","tagName":"documentação","confirm":true}
```

```
{"spaceId":"90070363783","tagName":"QA / revisão","confirm":true}
```

`clickupfy_task_remove_tag`

Remove uma tag da tarefa e mantém a definição no Space. Retorna `{ removed, taskId, tagName }`. As outras associações da tarefa permanecem intactas.

`taskId` e `tagName` são obrigatórios. `customTaskId` é booleano opcional, desativado por padrão; IDs personalizados usam o workspace do perfil.

CAMPO	TIPO	OBRIGATÓRIO	USO
`account`	string	não	Perfil a resolver.
`taskId`	string	sim	Tarefa que perderá a tag.
`tagName`	string	sim	Nome da tag.
`customTaskId`	booleano	não	Resolve o ID no workspace do perfil.

Exemplos:

```
{"taskId":"86abc123","tagName":"bug"}
```

```
{"taskId":"86abc123","tagName":"em revisão"}
```

```
{"taskId":"DEV-42","tagName":"backend","customTaskId":true}
```

```
{"account":"produto","taskId":"77def456","tagName":"documentação"}
```

```
{"taskId":"sub-1","tagName":"QA / revisão"}
```

Após uma escrita, consulte `clickupfy_tags_list` para conferir o catálogo ou `clickupfy_task_get` com `raw: true` para conferir as associações da tarefa.

Consulte a referência do CLI

Sintaxe global

```
clickupfy [options] <command> [subcommand] [arguments]
```

Sem argumentos, em um terminal interativo, o binário abre a TUI. Sem TTY, imprime o help e sai com exit code 2.

O comando de manutenção da instalação usa esta forma:

```
clickupfy upgrade [alvo]
```

As opções globais valem em qualquer posição, antes ou depois do comando:

OPÇÃO	EFEITO
`-V`, `--version`	Mostra a versão.
`-h`, `--help`	Mostra o help do comando atual.
`--account <perfil>`	Usa outro perfil somente nessa execução.
`--json`	Imprime a resposta no envelope `{ ok, command, data }`.
`--skill [tópico]`	Imprime a documentação embutida, inteira ou por tópico.
`--opencli`	Imprime a descrição OpenCLI da árvore de commands em JSON.
`--no-color`	Desliga cores ANSI. `NO_COLOR=1` tem o mesmo efeito.

Os tópicos de `--skill` são `cli`, `tui`, `mcp`, `config`, `exit-codes` e `reference`.

No exemplo abaixo, a resposta de `task get` usa o perfil `cliente-a` e sai em JSON:

```
clickupfy --account cliente-a --json task get 86abc123
```

Saída em texto e em JSON

Texto é o padrão quando `output.format` em settings.json é `text`. Com `--json`, a resposta segue este envelope:

```
{
  "ok": true,
  "command": "clickupfy task get",
  "data": { "task": {}, "execution": {} }
}
```

Em erro:

```
{
  "ok": false,
  "command": "clickupfy task get",
  "error": {
    "code": 6,
    "name": "NOT_FOUND",
    "message": "Task \"x\" not found.",
    "details": {}
  }
}
```

Listas vêm dentro de um objeto, como `data.tasks`, `data.spaces` ou `data.comments`. O exit code do processo é sempre o mesmo de `error.code`.

Configuração e identidade

```
clickupfy setup [options]
clickupfy status
clickupfy whoami
clickupfy account list
clickupfy account show [perfil]
clickupfy account use <perfil>
clickupfy account remove <perfil>
clickupfy account import [--force]
clickupfy workspace list
clickupfy workspace use <workspace-id>
clickupfy doctor [--online]
clickupfy config list
clickupfy config get <key>
clickupfy config set <key> <value>
clickupfy config unset <key>
clickupfy config path
clickupfy config schema
clickupfy credentials status
```

O setup recebe a credencial, identifica o perfil local e associa o workspace. `install` é o alias de `setup`. Estes parâmetros permitem executar o mesmo processo com ou sem prompts:

OPÇÃO	EFEITO
`--api-key <chave>`	Informa a API key pessoal do ClickUp.
`--token <chave>`	Mantém compatibilidade como alias de `--api-key`.
`--name <nome>`	Define o nome local do perfil, normalizado em kebab-case.
`--workspace <id>`	Associa o perfil ao workspace indicado.
`--non-interactive`	Executa sem abrir prompts e exige os valores necessários por opção.

`account remove <perfil>` aceita `--yes` para confirmar a remoção sem prompt. `account import` aceita `--force` para substituir perfis já existentes. As outras ações desse grupo não possuem opções próprias.

`clickupfy doctor` verifica localmente o runtime, o terminal, as permissões da pasta, o `settings.json`, a origem da master key, o cofre, o `~/.clickupfy/accounts.json`, o perfil resolvido, a presença da API key, o config antigo do ClickUpfy 0.9.0 e os arquivos `.mcp.json` e `.codex/config.toml` do projeto quando existirem. Use `--json` para automação. O comando não faz chamada ao ClickUp e não cria arquivos.

`clickupfy doctor --online` também valida a API key em `GET /user` e lê as skills instaladas com `npx skills list --json`. Essas verificações podem chamar a rede e baixar o gerenciador de skills.

Hierarquia

```
clickupfy space list [--archived]
clickupfy folder list --space <id> [--archived]
clickupfy list list --folder <id> [--archived]
clickupfy list list --space <id> [--archived]
clickupfy list get <list-id>
```

`space list`, `folder list` e `list list` aceitam `--archived`. O comando `folder list` exige `--space <id>`. Em `list list`, informe `--folder <id>` ou `--space <id>`, pois os dois destinos são mutuamente exclusivos e a resposta precisa pertencer a um único nível da hierarquia.

Tarefas

```
clickupfy task list --list <list-id>
clickupfy task search --query <texto>
clickupfy task get <task-id>
clickupfy task get <task-id> --markdown
clickupfy task get <task-id> --raw
clickupfy task create [options]
clickupfy task update <task-id> [options]
clickupfy task delete <task-id> [--yes]
clickupfy task add-tag <task-id> <tag-name>
clickupfy task remove-tag <task-id> <tag-name>
```

Filtros de leitura

COMANDO	OPÇÃO	EFEITO
`task list`	`--list <id>`	Define a List consultada e é obrigatório.
`task list`	`--status <valores>`	Restringe a resposta aos status informados.
`task list`	`--assignee <valores>`	Restringe a resposta aos responsáveis informados.
`task list`	`--include-closed`	Inclui tarefas concluídas.
`task list`	`--page <n>`	Consulta uma página, começando em zero.

COMANDO	OPÇÃO	EFEITO
`task search`	`--query <texto>`	Procura no nome, na descrição ou no ID.
`task search`	`--status <valores>`	Restringe a busca aos status informados.
`task search`	`--assignee <valores>`	Restringe a busca aos responsáveis informados.
`task search`	`--include-closed`	Inclui tarefas concluídas.
`task search`	`--max-pages <n>`	Limita a paginação consultada entre um e 100.
`task get`	`--raw`	Retorna a resposta original do ClickUp.
`task get`	`--markdown`	Concatena tarefa, subtarefas e checklists em Markdown.

`--status` e `--assignee` aceitam vários valores separados por vírgula, como em `--status "to do,review"` e `--assignee 7,8`.

`--raw` e `--markdown` não podem ser usados juntos. As duas opções também são incompatíveis com `--json`, e o CLI encerra a execução com uma mensagem de uso quando encontra a combinação.

Campos de criação

OPÇÃO	EFEITO
`--list <id>`	Define a List de destino e é obrigatório.
`--name <nome>`	Define o nome e é obrigatório.
`--description <texto>`	Envia uma descrição em texto.
`--markdown-content <markdown>`	Envia uma descrição preservada como Markdown.
`--status <status>`	Define o status inicial.
`--priority <n>`	Define prioridade de `1` a `4`.
`--assignee <ids>`	Informa IDs dos responsáveis, separados por vírgula.
`--parent <task-id>`	Cria uma subtarefa ligada ao item informado.
`--start-date <AAAA-MM-DD>`	Define a data de início.
`--due-date <AAAA-MM-DD>`	Define a data de entrega.
`--points <n>`	Define Sprint Points com número igual ou maior que zero.

Datas são gravadas no início do dia no fuso local da máquina, para o ClickUp mostrar só a data. Escolha `--description` para texto ou `--markdown-content` para Markdown. Evite enviar os dois campos na mesma criação para não deixar a representação da descrição ambígua.

Campos de atualização

OPÇÃO	EFEITO
`--name <nome>`	Altera o nome.
`--description <texto>`	Substitui a descrição por texto.
`--markdown-content <markdown>`	Substitui a descrição por Markdown.
`--status <status>`	Altera o status.
`--priority <n>`	Altera a prioridade de `1` a `4`.
`--start-date <AAAA-MM-DD>`	Altera a data de início.
`--clear-start-date`	Remove a data de início.
`--due-date <AAAA-MM-DD>`	Altera a data de entrega.
`--clear-due-date`	Remove a data de entrega.
`--points <n>`	Altera os Sprint Points.

O comando exige pelo menos um campo. Quando uma data e sua opção `--clear-*` aparecem na mesma execução, a remoção prevalece. `task delete` usa `--yes` para confirmar a exclusão sem prompt.

Tags

```
clickupfy tag list --space <space-id>
clickupfy tag create <name> --space <space-id> --foreground "#ffffff" --background
"#336699"
clickupfy tag update <tag-name> --space <space-id> --name <novo-nome>
clickupfy tag delete <tag-name> --space <space-id> --yes
```

`--foreground` e `--background` aceitam cores `#RRGGBB`. `task add-tag` e `task remove-tag` aceitam `--custom-task-id` para trabalhar com o id personalizado do workspace.

Depois de escolher os campos, consulte a ajuda da versão instalada para comparar a referência com o contrato executável:

```
clickupfy task create --help
clickupfy task update --help
```

Checklists e comentários

```
clickupfy checklist create <task-id> --name <nome>
clickupfy checklist item-create <checklist-id> --name <nome>
clickupfy checklist set \
  <task-id> <checklist-id> <item-id> \
  --resolved
clickupfy checklist set \
  <task-id> <checklist-id> <item-id> \
  --open
clickupfy comment list --task <task-id>
clickupfy comment create --task <task-id> --text <texto>
```

`checklist item-create` aceita `--assignee <id>` para associar o item a um responsável. Em `checklist set`, escolha exatamente `--resolved` ou `--open` para que a releitura confirme o estado pretendido. `comment create` aceita `--notify-all` quando a atualização precisa notificar os participantes da tarefa.

Sprints

```
clickupfy sprint list --folder <folder-id>
clickupfy sprint current --folder <folder-id>
clickupfy sprint get <sprint-id>
clickupfy sprint tasks <sprint-id> [--open-only]
clickupfy sprint add-task <sprint-id> <task-id>
clickupfy sprint remove-task <sprint-id> <task-id>
clickupfy sprint set-points <task-id> <points>
```

Os comandos de Sprint aceitam opções de calendário, inclusão e estado para separar uma List comum do ciclo que será analisado:

COMANDO	OPÇÃO	EFEITO
`sprint list`	`--archived`	Inclui Lists arquivadas.
`sprint list`	`--include-regular`	Inclui Lists sem período do Sprint Folder.
`sprint list`	`--at <AAAA-MM-DD>`	Calcula o estado do ciclo na data informada.

COMANDO	OPÇÃO	EFEITO
`sprint current`	`--at <AAAA-MM-DD>`	Procura a Sprint ativa na data informada.
`sprint get`	`--at <AAAA-MM-DD>`	Calcula o relatório na data informada.
`sprint tasks`	`--open-only`	Omite tarefas concluídas.

`sprint list` e `sprint current` exigem `--folder <id>` para limitar a busca ao Sprint Folder selecionado.

Time tracking

```
clickupfy time current
clickupfy time start --task <task-id> [--description <texto>]
clickupfy time stop
```

`time start` com outro cronômetro ativo em outra tarefa troca o cronômetro, avisa no stderr e devolve a tarefa anterior em `previous`.

Docs

```
clickupfy doc list [--query <texto>] [options]
clickupfy doc get <doc-id>
clickupfy doc create --name <nome> [options]
clickupfy doc page tree <doc-id>
clickupfy doc page list <doc-id> [options]
clickupfy doc page get <doc-id> <page-id> [--content-format <valor>]
clickupfy doc page create <doc-id> --name <nome> [options]
clickupfy doc page update <doc-id> <page-id> [options]
```

Docs usam a API v3 do ClickUp e sempre operam no workspace do perfil ativo, sem depender de Space, Folder ou List.

COMANDO	OPÇÃO	EFEITO
`doc list`	`--query <texto>`	Filtra pelo nome ou ID localmente.
`doc list`	`--parent-id <id>`	Restringe aos Docs criados sob este local.
`doc list`	`--parent-type <n>`	Tipo do local: `4` Space, `5` Folder, `6` List, `7` Everything, `12` tarefa.
`doc list`	`--deleted`	Inclui Docs excluídos.

COMANDO	OPÇÃO	EFEITO
`doc list`	`--archived`	Inclui Docs arquivados.
`doc list`	`--creator <id>`	Restringe ao ID numérico do criador.
`doc list`	`--max-pages <n>`	Limita a paginação por cursor, entre uma e 50 páginas.
`doc create`	`--parent-id <id>`	Local onde o Doc nasce; exige `--parent-type`.
`doc create`	`--parent-type <n>`	Tipo do local; exige `--parent-id`.
`doc create`	`--visibility <valor>`	`PRIVATE` ou `PUBLIC`.
`doc create`	`--create-page`	Cria também a primeira página em branco.
`doc page list`	`--max-page-depth <n>`	Limita a profundidade de sub-páginas retornadas.
`doc page list` / `doc page get`	`--content-format <valor>`	`text/md` (padrão) ou `text/plain`.
`doc page create`	`--content <texto>`	Conteúdo inicial da página.
`doc page create`	`--file <caminho>`	Lê o conteúdo de um arquivo; `--file -` lê do stdin.
`doc page create`	`--sub-title <texto>`	Subtítulo da página.
`doc page create`	`--parent-page <id>`	Cria como sub-página deste ID.
`doc page create`	`--orderindex <n>`	Posição entre as páginas irmãs.
`doc page update`	`--content-edit-mode <modo>`	`replace` (padrão), `append` ou `prepend`.

`doc page tree` mostra a hierarquia de páginas sem conteúdo, o caminho mais rápido para localizar um `page-id`. `doc page update` exige pelo menos um campo entre `--name`, `--sub-title`, `--content` e `--file`. `--content` e `--file` juntos saem com exit code 2.

A API pública do ClickUp não oferece endpoints para excluir Docs ou páginas, reordenar páginas na árvore, nem gerenciar permissões de compartilhamento.

API genérica

```
clickupfy api list
clickupfy api describe <operation>
clickupfy api call <operation> [options]
```

O catálogo cobre as 173 operações das especificações públicas v2 e v3. O argumento `<operation>` aceita o `operationId` (`GetTask`), o nome de CLI (`v2-get-task`) ou o nome da tool (`clickupfy_api_v2_get_task`).

```
clickupfy api list --group Tasks
clickupfy api describe GetTask
clickupfy api call GetTask --path '{"task_id":"abc123"}'
clickupfy api call v2-create-task-attachment --path '{"task_id":"abc123"}' --file
attachment[0]=./relatorio.pdf
```

`api call` aceita `--path` , `--query` , `--body` , `--body-file` , `--input` , `--content-type` , `--file` , `--yes` e `--dry-run` . Operações `DELETE` pedem confirmação no terminal; fora dele, sem `--yes` , saem com exit code 2.

Agentes e MCP

```
clickupfy agent skill list
clickupfy agent skill show <nome>
clickupfy agent skill install [skills...] [--global] [--force]
clickupfy agent install [options]
clickupfy mcp setup [options]
clickupfy mcp [--list] [--list-id <id>] [options]
clickupfy mcp serve --list <id> [options]
```

`agent skill install` aceita uma lista opcional de skills e chama `npx skills add promovaweb/clickupfy --copy --agent codex --skill <nome>...` . Sem `--global` , o gerenciador usa `.agents/skills/` e cria ou atualiza `skills-lock.json` no projeto. Com `--global` , usa o catálogo global do usuário. Para consultar o estado real, use `npx skills list` ou `npx skills list --global` ; `clickupfy agent skill list` exibe somente o catálogo embutido no binário.

O comando não oferece `--target` : o gerenciador de skills controla os caminhos canônicos. `--force` permanece aceito por compatibilidade.

`agent install` exige `--space <id>` e `--list <id>` . Ele também aceita `--global` , `--workspace <id>` , `--folder <id>` , `--sprint-folder <id>` e `--force` . O comando atualiza o servidor `promovaweb-clickupfy` no `.mcp.json` e no `.codex/config.toml` , preservando as outras entradas de cada arquivo. `agent init` é o alias.

`mcp setup` grava uma entrada fixada nos quatro formatos de cliente e aceita `--name` , `--space` , `--list` , `--clients` , `--project` , `--read-only` , `--force` e `--non-interactive` .

`mcp serve` exige `--list <id>` . `mcp` aceita `--list-id <id>` . Os dois aceitam `--account <perfil>` , `--workspace <id>` , `--space <id>` , `--folder <id>` , `--sprint-folder <id>` e `--read-only` .

Manutenção e outros modos

```
clickupfy
clickupfy tui --tab about
clickupfy about
clickupfy upgrade
clickupfy skill cli
clickupfy completion bash
```

`clickupfy` sem argumentos abre a TUI na aba Overview, com o perfil, o workspace e o cronômetro em execução. A tecla `r` atualiza os dados.

`clickupfy --skill` imprime a documentação embutida. `clickupfy completion` gera o script para `bash`, `zsh` ou `fish`.

Exit codes

CODE	NAME	QUANDO
`0`	`SUCCESS`	operação concluída
`1`	`GENERIC_ERROR`	falha inesperada sem classificação
`2`	`USAGE`	arguments, options ou subcommand inválidos
`3`	`CONFIG`	settings.json ausente, inválido ou key desconhecida
`4`	`CREDENTIALS`	cofre ilegível, master key ausente ou credencial não encontrada
`5`	`NETWORK_OR_API`	falha de rede ou resposta de API fora do esperado
`6`	`NOT_FOUND`	recurso solicitado não existe
`7`	`PERMISSION`	permissão negada no sistema de arquivos
`8`	`NO_TTY`	a TUI exige um terminal interativo
`9`	`MCP`	o servidor MCP falhou ao iniciar ou perdeu o transporte
`130`	`INTERRUPTED`	entrada cancelada ou processo interrompido com Ctrl+C

Escolha o formato de saída

A tabela compacta é adequada para leitura humana. Use `--json` quando um script precisar de campos completos ou quando a tabela não mostrar um detalhe da API. Listas vêm dentro de um objeto, como `data.tasks`.

Use `task get --markdown` para contexto textual. Use `--raw` apenas quando precisar comparar a resposta original do ClickUp com a projeção do ClickUpfy.

Ajuda é a referência da versão instalada

Esta documentação explica o contrato estável. Para flags exatas da versão presente na máquina:

```
clickupfy --help
clickupfy <grupo> --help
clickupfy <grupo> <ação> --help
```

Quando a ajuda estiver correta, mas a execução falhar, continue no guia de solução de problemas para conferir instalação, credencial, escopo e resposta da API.

Resolva falhas comuns

O comando não foi encontrado

Executado pelo npm, confira o diretório global:

```
npm prefix --global
```

Confirme se a pasta de executáveis globais está no `PATH`. Em instalação por binário avulso, confira permissão de execução e o nome do arquivo.

```
chmod +x clickupfy
clickupfy --version
```

A API key foi recusada

Use:

```
clickupfy whoami
```

Se falhar, gere ou confira a chave no ClickUp e execute `clickupfy setup` de novo. Uma chave revogada não pode ser recuperada pelo ClickUpfy.

Nunca publique a chave para pedir suporte. Informe apenas o perfil, o workspace mascarado quando necessário e a mensagem de erro sem headers.

O workspace está incorreto

```
clickupfy workspace list
clickupfy workspace use <workspace-id>
clickupfy status
```

Se o MCP fixa `--workspace`, atualize a configuração do projeto para o mesmo ID ou selecione o perfil correto. Um workspace diferente do perfil é recusado pelo servidor.

Folder ou List não aparece

Confirme o Space e tente incluir arquivados:

```
clickupfy folder list --space <space-id> --archived
clickupfy list list --folder <folder-id> --archived
```

Para List sem Folder, use `--space`. Permissões da API key também podem ocultar recursos.

O status foi rejeitado

```
clickupfy list get <list-id>
```

Copie a grafia de um status retornado. Status são configuráveis por List.

O MCP não inicia

Execute manualmente o comando registrado no `.mcp.json` ou no `.codex/config.toml`. Os motivos mais comuns são:

- `clickupfy` ausente no `PATH` do cliente
- JSON ou TOML inválido
- perfil inexistente
- `--list` ou `--list-id` sem valor
- workspace fixado diferente do perfil
- cliente não reiniciado depois da edição.

Rode o diagnóstico local antes de abrir o cliente:

```
clickupfy doctor
clickupfy doctor --online
```

O MCP recusou um ID

Essa recusa é esperada quando a ferramenta recebe um ID diferente do escopo do projeto. Consulte:

```
clickupfy_mcp_context
```

Use o destino fixado ou altere o arquivo do cliente conscientemente. Não contorne a proteção repetindo a chamada com outro recurso.

Uma ferramenta de escrita não aparece

O servidor pode estar em read-only. Confira os argumentos. Remover `--read-only` amplia as permissões do cliente e deve ser uma decisão explícita.

A Sprint atual não foi encontrada

Liste o Folder com períodos:

```
clickupfy sprint list --folder <sprint-folder-id> --include-regular
```

Confirme se existe exatamente uma Sprint cujo período inclui a data atual. Corrija datas ou sobreposição na interface do ClickUp.

O item de checklist não mudou

Releia a tarefa:

```
clickupfy task get <task-id> --json
```

Confirme task ID, checklist ID e item ID. O comando `checklist set` rejeita um item que não pertença à tarefa informada.

O time entry já está em execução

```
clickupfy time current
clickupfy time stop
```

Confira o item antes de encerrar. Depois inicie o registro correto.

A saída compacta não mostra um campo

Repita com `--json`:

```
clickupfy task get <task-id> --json
```

Leia o campo dentro de `data`, como `data.task` ou `data.tasks`. Para uma tarefa longa consumida por agente, prefira `--markdown`.

Um script antigo quebrou depois da migração

O `--json` da versão 1.0 usa o envelope `{ ok, command, data }`. Scripts que liam o objeto cru passam a ler `.data`, e o código de erro fica em `error.code`. Veja a referência do CLI e a tabela de exit codes no help.

Diagnóstico seguro

Ao relatar um problema, inclua:

- versão de `clickupfy --version`
- plataforma e forma de instalação
- grupo e ação executados
- mensagem de erro
- se o modo era CLI, TUI ou MCP
- escopo sem credenciais.

Remova API keys, tokens, headers, conteúdo de clientes e dados pessoais.

Volte ao início do guia ou consulte a referência do CLI.